

Chapter 7

Design for Test by Means of Scan

Most test generation schemes look at a CUT as a black box, the only available nodes of which for testers to control are its primary inputs, and to observe one are its primary outputs. This limited controllability and observability of circuits under test (CUT) means complex test generation algorithms for combinational circuits, and near-impossible test generation for the sequential circuits.

To overcome this difficulty in testing, digital circuits must become more testable by incorporation of design for test (DFT) techniques. For this purpose, designers must get involved in the test process by incorporating testability hardware in their designs and evaluating their designs for testability. DFT techniques offer ways of making internal structure of a design more controllable and easier to observe. Because such tasks are handled by designers, hardware description languages play an important role in facilitating insertion and evaluation of hardware structures that are put in a circuit for making it more testable.

This chapter discusses DFT techniques. We begin by covering some of the basics, followed by a section on options we have for hardware structures that make a circuit more testable. This section leads to some of the established scan insertion techniques. The section that follows this highlights the use of HDLs in modeling DFT techniques, evaluating them, and verifying their operation.

7.1 Making Circuits Testable

A circuit is testable if tests can be generated for it efficiently, it can be tested with a high fault coverage, and the time it takes to test the manufactured part is reasonable. Testability is a combination of controllability and observability. A circuit becomes more testable by making it more controllable and more observable [1, 2].

7.1.1 Tradeoffs

Improvements in controllability and observability are almost always done by inserting additional hardware in a design. This means that DFT techniques that make circuits more testable always introduce additional hardware that results in more pins, more delays, more power consumption, and obviously a hardware overhead.

What we are getting instead is a better coverage and reduced test time, and in some cases, DFT techniques gain access to the internal structures of a circuit that would otherwise be impossible to test.

Reducing the time it takes to test a manufactured part is the most important concern of DFT techniques. This goal is achieved by introducing extra test hardware in the CUT. Reducing this hardware, its power consumption during test, the delay it introduces in the design, and the extra pins that are needed to take test to the manufactured part are the parameters a design engineer must optimize.

7.1.2 Testing Sequential Circuits

Perhaps one of the most important contributions of DFT is making sequential circuits testable. The previous chapter showed how complex it is to generate a complete test for a sequential circuit. It also showed that even if tests are generated, application of test requires many clock cycles to move a circuit into states that activate faults.

DFT techniques alter a sequential circuit model in such a way that combinational test techniques can be used for it. With this alteration, test generation schemes, random or deterministic, can be used to generate test for the altered circuit. DFT techniques define ways in which tests generated for a combinational model of a sequential circuit can be applied to the actual sequential circuit [3].

7.1.2.1 Sequential Circuit Huffman Model

Huffman model, as discussed in Chap. 2, is a useful model for sequential circuit testing. As shown in Fig. 7.1, a sequential circuit is modeled by a combinational circuit having feedback through delay elements. In synchronous sequential circuits, the delay elements are clocked flip-flops, and each feedback is a state variable of the circuit.

Circuit primary inputs and primary outputs only apply to the combinational part. Synchronous or asynchronous set, reset, and other control inputs only apply to the feedback flip-flops. This model completely separates a CUT's registers from its combinational part. The combinational part consists of all circuit buses, logic units, arithmetic functions, and discrete logic parts. The feedback registers consist of control flip-flops, data registers, and small register files, including register signals such as set, reset, enable, and parallel load.

Although memory parts of a system can also fit in this model, for test purposes, this is not recommended. Memories must be separated from logic parts of a system, and memory buses (data and address) must be treated as inputs and outputs of the above sequential circuit model.

7.1.2.2 Unfolding Sequential Model

As discussed above, the usefulness of the above model is separation of registers (sequential part) and combinational part of a digital system. For test purposes, this separation enables application of test methodologies to the combinational part of the circuit, and treating registers separately.

Since the majority of a circuit's logic gates are contained in the combinational part, testing this part, while considering register part inputs and outputs, covers the majority of a circuit's faults.

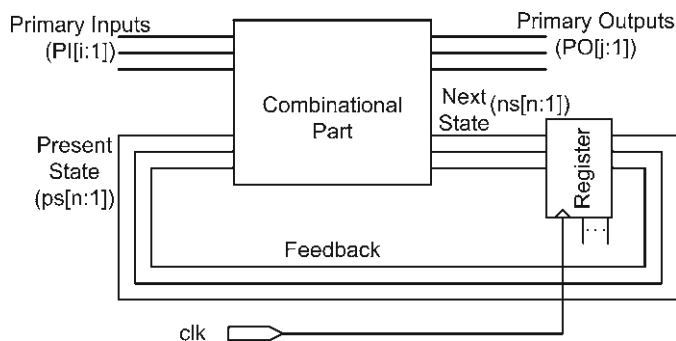


Fig. 7.1 Huffman model for test

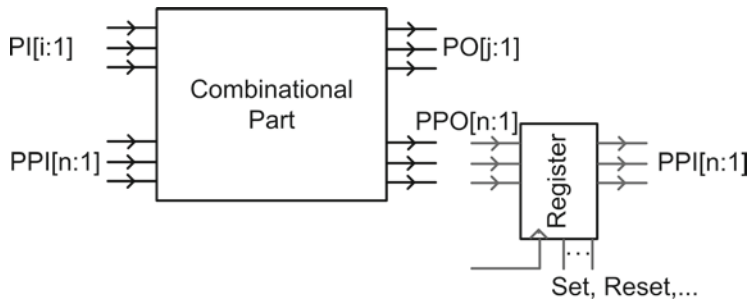


Fig. 7.2 Unfolded circuit model

Also, since flip-flops are treated as primitive building blocks in most technologies, testing a circuit for internal flip-flop faults is a secondary issue in most logic test systems.

A circuit model that separates combinational and register parts of a digital system and puts the focus of testing on the combinational part is obtained by unfolding the circuit model of Fig. 7.1. As shown in Fig. 7.2, by unfolding a sequential circuit, its registers are separated and ignored. Register outputs become pseudo inputs for the unfolded circuit, and register inputs become pseudo outputs of this circuit.

For test purposes, only the upper block in Fig. 7.2 is considered. Fault collapsing, test generation, and evaluation of test vectors are all done on this circuit model. Note that faults on pseudo inputs and pseudo outputs (PPI, PPO) of the unfolded model are the same as those of the feedback register outputs and inputs, and testing the unfolded model already includes testing the register ports.

Testing register clock, set, reset, etc. are not dealt with when considering the unfolded model and must be treated separately. Test results obtained from the unfolded model are used by test equipment for testing the actual circuit. A Verilog testbench that models this test equipment or test environment of the actual circuit is called a virtual tester. Section 7.4 elaborates on this topic.

7.1.3 Testability of Combinational Circuits

Combinational circuits can also benefit from DFT techniques. With an additional hardware inserted into a combinational circuit, it can be made to give a better coverage, reduce the number of test vectors required to test it, or achieve both at the same time. DFT techniques alter a combinational circuit for better controllability and observability. The model of the altered circuit is used for test generation.

7.2 Testability Insertion

By simply adding a jumper, or an extra input or output pin, we have taken steps in making our circuit more testable. This obviously adds additional hardware to our circuit, and the use of such testability provisions must carefully be planned. Limit on the number of extra input/output pins for testing, gained test time, ease of testing, and issues such as timing and power consumption must be considered when choosing DFT techniques.

This section discusses various choices that we have for hardware structures for making a circuit more testable [4]. In general, this section shows how a “test point” can be added to a circuit and how

the associated hardware with test points can be minimized and/or shared with other test points. We start with a simple hardware for forcing a 0 or a 1 on a line, and progressively get into multiplexer and register structures for reducing our hardware overhead.

7.2.1 Improving Observability

Improving observability always helps testability of a circuit. Adding output pins improves observability, and enables fault effects that would otherwise not propagate to a primary output have a chance to show themselves through newly added output pins. The end result is fewer test vectors to test for faults of a CUT, thus less test application time for the final manufactured chip.

Line observability values are measured by SCOAP or are probability based. Lines with observability values below a certain threshold (higher SCOAP parameter values) can be considered as candidates for becoming extra output pins.

Improving testability by means of observability can also be achieved by making state flip-flop outputs observable by pulling them out as primary outputs. Figure 7.3 shows the Mealy machine implementation of the previous chapter in which flip-flop outputs are pulled as primary outputs. Starting in state 10, while $a = 1$, the SA0 fault shown can be detected in one clock cycle on the new $Out V_1$ output. Provisions for forcing this circuit into a certain state (e.g., state 10) are made in Fig. 7.3. Such techniques include 0 or 1 insertion and will be discussed in the next sub section.

In an RT-level design, control signals going out from circuit's controller and controlling flow of data through buses and logic units, as well as logic and ALU control inputs are appropriate places where extra output pins can be added.

Other candidates for observing via new circuit outputs are bus and multiplexer select inputs, multiplexer and decoder enable inputs, tri-state controls, register load input, count up and count down signals, shift-register mode control inputs, and feedback lines.

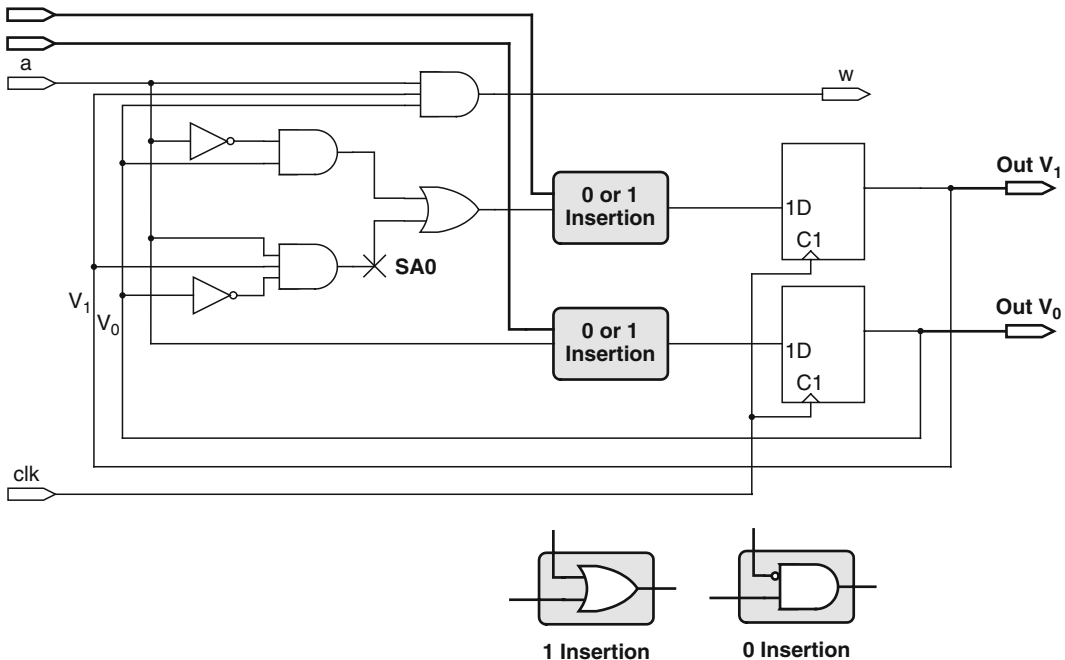


Fig. 7.3 Basic testability techniques (observability, controllability)

7.2.2 Improving Controllability

Controllability parameters identify hard to reach places in a combinational circuit. As a first step in adding controllability to a combinational circuit, low controllability lines should be considered. In addition, by directly controlling fanout stems and branches from a circuit's inputs, problems in test generation contributed by reconvergent fanouts can be resolved.

In sequential circuits, flip-flop inputs are good candidates for being directly controlled by circuit inputs. In addition, controlling flip-flop reset and other control inputs help driving a sequential circuit into a given state.

In RT-level designs, direct control of control signals coming from a circuit's controller to its datapath enables testing of individual data operations independently. For example, being able to externally control the *load accumulator* control signal of a datapath enables testing the loading of the accumulator register and the bus structures that drive it.

Other places where external control can contribute to testability of a circuit include multiplexer and bus control inputs, tri-state control inputs, arithmetic and logic unit select inputs, and counter and shift-register mode control signals.

Unlike observability that can easily be achieved by adding the line to be observed to our primary outputs, there are several ways controllability can be achieved. Figure 7.4 shows several such methods. Shown here are several hardware structures that intercept a line that would normally go between two logic parts. The hardware inserted in the line between *Logic A* and *Logic B* uses primary inputs to control value going to *Logic B*.

Figure 7.4a shows 0-insertion logic. The line labeled *Insert 0* is the controlling line that can be directly driven by a primary input. The circuit is in its normal mode of operation when *Insert 0* is 0. To drive a 0 into the input of *Logic B*, *Insert 0* must be set to 1. Figure 7.4b shows a 1-insertion logic

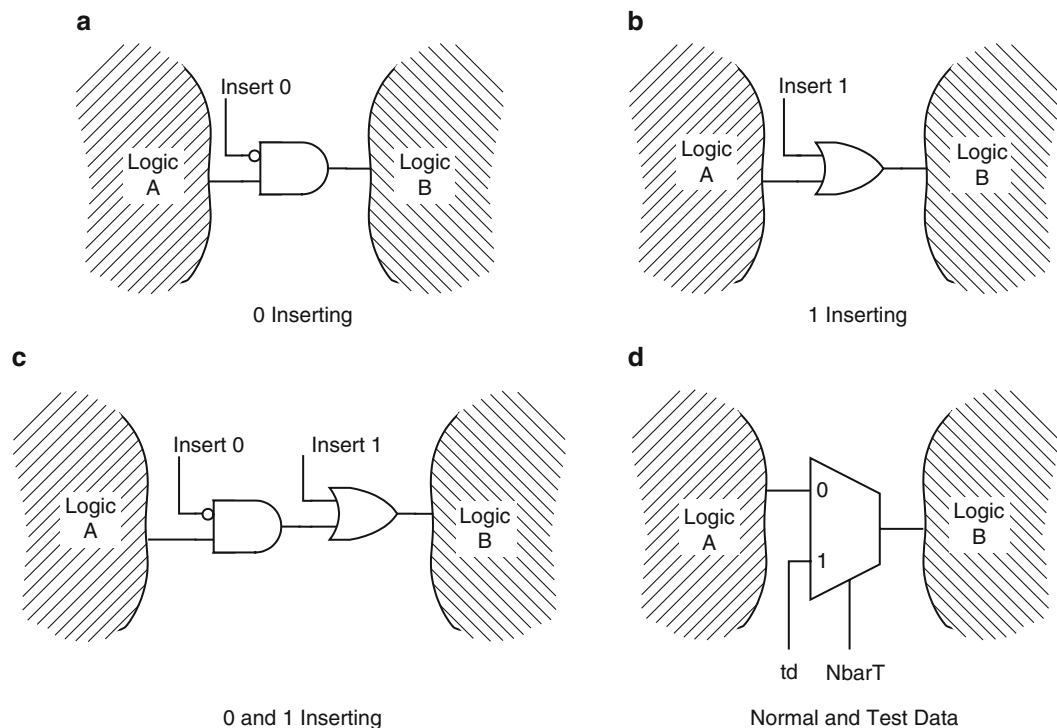


Fig. 7.4 Adding controllability

that works like the 0-insertion, except that asserting *Insert 1* puts a 1 at the input of *Logic B*. For being able to control input of *Logic B* with either a 0 or a 1, logic shown in Fig. 7.4c should be used. In combinational circuit testing, 0-insertion and 1-insertion structures are useful in places with low 0- and 1-controllability, respectively.

Figure 7.4d shows a more general case of driving a line in a circuit with a given test data (*td*). With use of a multiplexer, signal *NbarT* puts the circuit in normal (when 0) or test (when 1) mode. In the test mode, instead of data from *Logic A*, *td* that can be driven by a primary input acts as an input to *Logic B*.

If hardware structures shown here are independent and do not share pins with other test structures, 0- and 1-insertions require one primary input pin, while the other two structures shown in Fig. 7.4 require two pins. In all cases shown, in addition to the gates used and the required input pins, timing delays are also added between the two logic parts. The multiplexer (Fig. 7.4d) has the largest delay and is the most general case.

Figure 7.3 that we used for demonstrating improvement in observability also shows controllability hardware added to the flip-flop inputs. Recall from the discussion in Sect. 7.2.1 that making flip-flop outputs observable makes testing for the fault shown possible by starting in state $V_1V_0 = 10$, and clocking the circuit once while *a* is 1. To force this state into the flip-flops, 1-insertion and 0-insertion structures should be used at the inputs of flip-flop 1 and flip-flop 0, respectively.

Testing circuit of Fig. 7.3 becomes easier by use of multiplexers at its flip-flop inputs. Figure 7.5 shows multiplexers at flip-flop inputs sharing the *NbarT* test control input. To force a given state into the circuit, *NbarT* should be driven with a 1 and the desired state value should be placed on *Td₁* and *Td₀* test data inputs.

7.2.3 Sharing Observability Pins

Section 7.2.1 discussed the use of observability points and showed an example where state flip-flops of a finite state machine were placed on external pins. To reduce the cost of external pins, a multiplexer can be used. As shown in Fig. 7.6, an *n*-to-1 multiplexer can be used to multiplex *n* test points into one output pin. This scheme requires $\log_2 n$ input pins for multiplex select inputs.

Multiplexing observation points in a circuit prevents simultaneous observation of multiple observation points, thus increases test time. In addition, the multiplexer adds extra hardware and delay overhead.

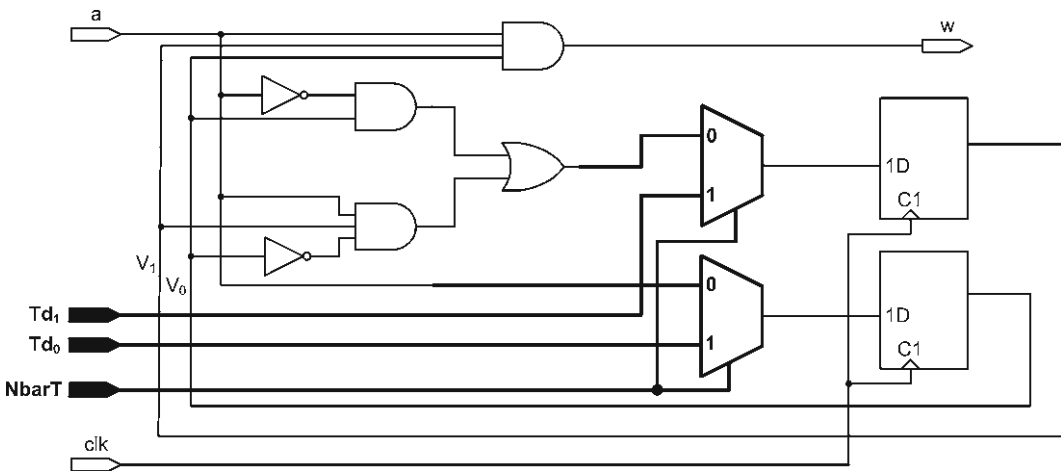
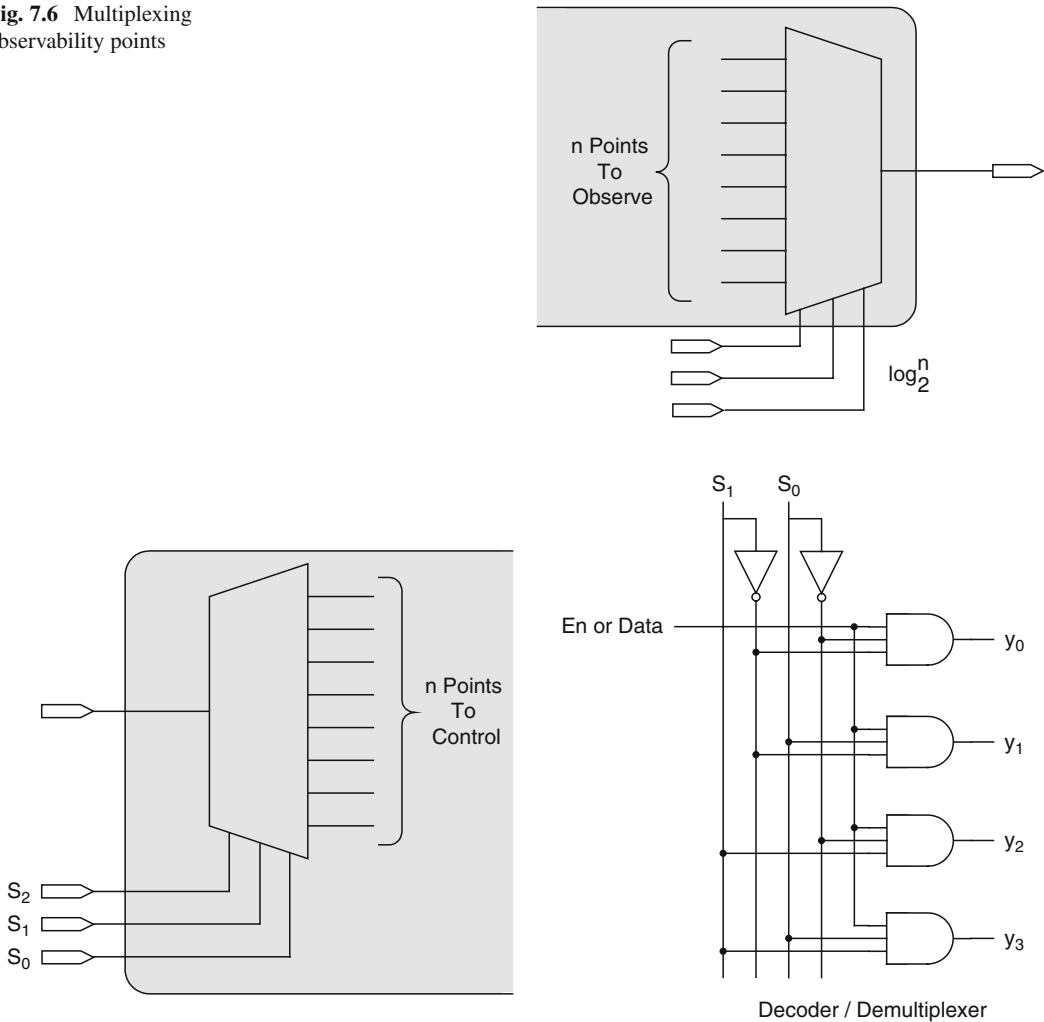


Fig. 7.5 Put circuit into any desired state

Fig. 7.6 Multiplexing observability points**Fig. 7.7** Demultiplexer/decoder, reduce pins for control

The hardware shown can be repeated if multiple activation of several observation points are required. For observing parallel buses, same bit positions of several buses to simultaneous can be grouped, and the hardware shown in Fig. 7.6 can be used for each group. In this case, the select inputs can still be shared, but the actual output data pin must be repeated for each group.

7.2.4 Sharing Control Pins

To reduce pins required for controlling n internal lines of a circuit, a 1-to- n demultiplexer is used. The demultiplexer has several select inputs that select the output that the data input drives.

A demultiplexer is a decoder with an enable input. Figure 7.7 shows a demultiplexer symbol and logic structure for a 1-to-4 demultiplexer, which is no different than a 2-to-4 decoder.

Figure 7.8 shows a 2-to-4 decoder used for controlling four 1-insertion lines. The decoder enable input is used as $NbarT$ input. When this input is 0, none of the decoder outputs is 1, and

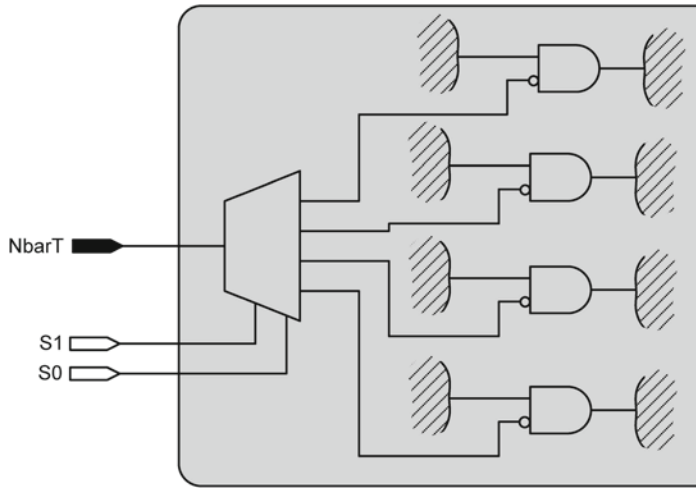


Fig. 7.8 Sharing 1-insertion hardware

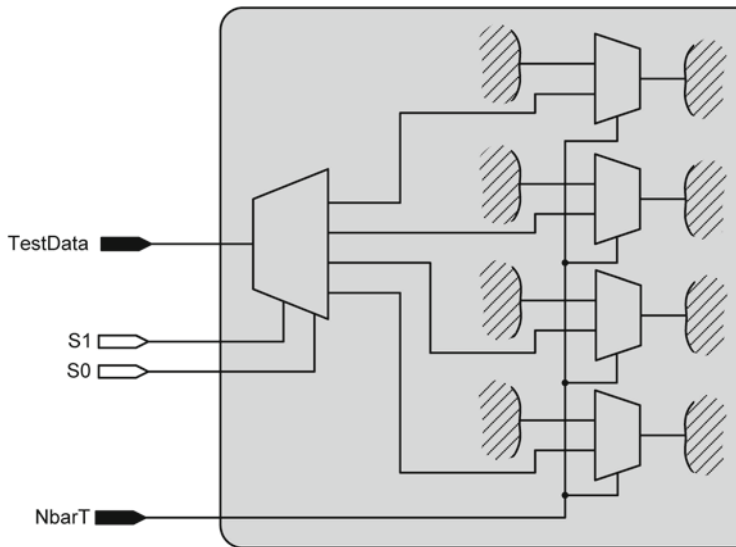


Fig. 7.9 Sharing test data insertion hardware

circuit works in normal mode. For test mode, $NbarT$ becomes 1, and according to value of $S1$, $S0$, one of the 1-insertion points becomes active. With this hardware, for controlling n lines of a CUT, $\log_2 n + 1$ pins are needed.

For multiple test data insertions (Fig. 7.4d), a demultiplexer helps reducing input pin requirements. Figure 7.9 shows a 1-to-4 demultiplexer providing test data for four multiplexers for test data insertions. With this hardware, for controlling n lines of a CUT, $\log_2 n + 2$ pins are required.

As shown in Fig. 7.9, when $NbarT$ is 0, the multiplexers between logic parts (shaded areas) connect the left logic to the right logic, and the test hardware is transparent. In the test mode ($NbarT = 1$), $S1$ and $S0$ select *TestData* for driving the input of one of the left-hand side logic blocks.

Demultiplexing control points prevents simultaneous assignment of values to the control points that share pins. Therefore, simultaneous assignment of test data to bits of parallel buses is not

possible unless the above hardware is repeated for each bus bit that needs to be controlled. In this case, select inputs (e.g., $S1$, $S0$) and $NbarT$ can still be shared by controllability hardware for different bus bits. Figure 7.10 shows extension of Fig. 7.9 for parallel buses. This figure shows $TestData0$ through $TestData_n$ providing test data for four n -bit buses, A , B , C , and D . Select inputs $S1$ and $S0$ select which bus the test data will go to.

7.2.5 Reducing Select Inputs

Although multiplexing observation points of Fig. 7.6 and demultiplexing control points of Fig. 7.7 significantly reduce test pin counts, for a large number of test points, we still face the problem of having many pins for multiplexer or demultiplexer select inputs. For the price of still longer test time, we can resolve this problem by adding a counter to the CUT. The counter clock and count inputs will be driven by external pins and the counter output selects point to be controlled or observed. For controlling or observing a line, the counter counts up, and when it reaches the number of the test point, other test signals will be issued. Figure 7.11 shows a counter used for selection inputs of a control point demultiplexer.

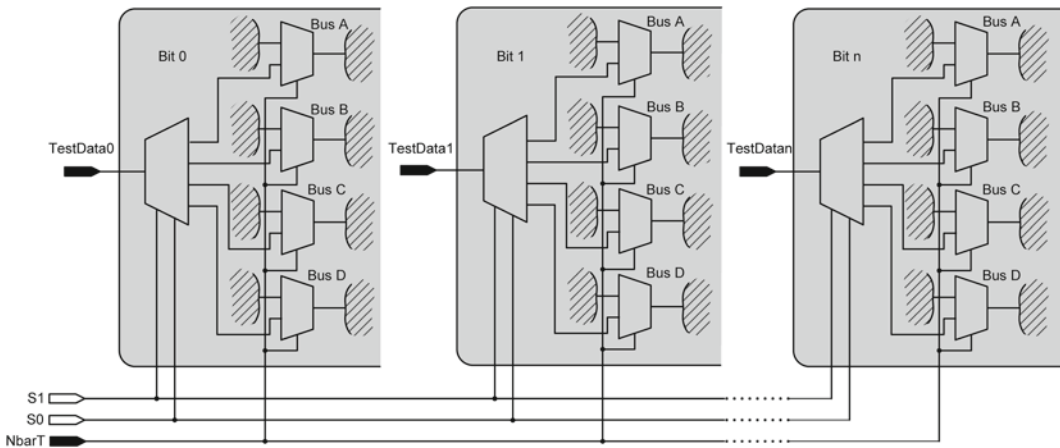


Fig. 7.10 Simultaneous control of bits of buses

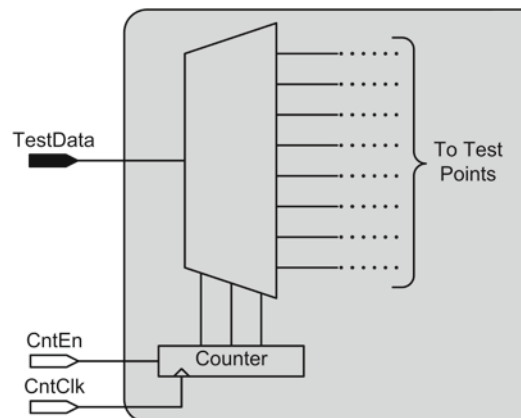


Fig. 7.11 Selection input counter

To select a test point, the counter counts up to the location of the test point on the multiplexer or demultiplexer. At the test application time, *TestData* drives the test point selected by the counter. The price we are paying for the reduction in the number of test pins is longer test time.

7.2.6 Simultaneous Control and Observation

DFT techniques discussed above were effective in reducing the number of test pins. On the other hand, none of these techniques allowed simultaneous activation of control or observe points, unless separate test pins were used and hardware structures were repeated (e.g., Fig. 7.10).

7.2.6.1 Simultaneous Control of Test Points

To allow simultaneous control for several test points and still keeping the number of pins low, a shift-register can be used to take test data serially, and apply all the bits in parallel in the test mode. Figure 7.12 shows a shift-register used for taking serial test data to be applied simultaneously to several logic blocks. Shift-register in this scheme replaces the demultiplexer of Fig. 7.9.

In the normal mode of operation, $NbarT$ is 0 and the multiplexers take the normal data from left logic blocks to the right ones. While the circuit is operating in this mode, test data can be shifted into the shift-register with the *TestClock* without affecting the CUT's normal operation. When test

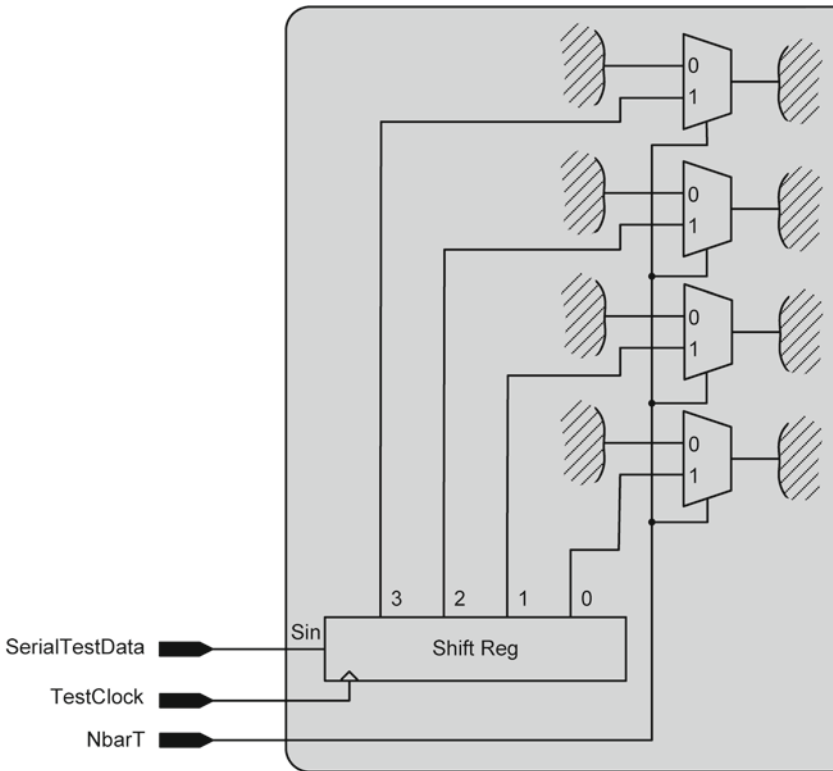


Fig. 7.12 Shift-register for simultaneous control

data are completely shifted in, the CUT is put in the test mode (i.e., $NbarT = 1$), which allows the test data from the shift-register to be used for input of logic blocks that are being tested (right-hatched blocks in Fig. 7.12).

Because test data shifting takes several clock cycles, we overlap the normal operation of CUT with shifting test data into the shift-register. In this case, when data are being shifted, $NbarT$ must be 0, so that changing test data at the shift-register output does not interfere with the normal data. In general, serial shifting of test data is a time-consuming process, but, as we will see later in this chapter, it is the only way testing can be done. One way to improve this is by use of a faster clock for the test clock. Since shift-register is the only recipient of the test clock, circuit delays do not apply, and this clock can be much faster than the normal CUT's clock.

7.2.6.2 Simultaneous Observation of Test Points

The concept of shift-registers mentioned above can also be used for simultaneously observing several lines in a CUT. Figure 7.13 shows a shift-register used for simultaneous collection of values from several test points and shifting them out serially. The device or ATE that is reading the output must perform serial to parallel conversion for having data from all test points at the same time.

In the test mode, when the response of CUT is ready and available at the shift-register input, the *load* input of the shift-register is enabled and the shift-register is clocked. This operation loads CUT's line values, or lines to be observed, into the shift-register. Following this operation, the shift-register *load* input is deasserted, which puts the shift-register in the serial mode. After application of n clocks in this mode, the line values being observed have been shifted out, and the shift-register is ready for the next parallel load of the test output data.

7.2.6.3 Isolated Serial Scan

The two concepts of using a shift-register for controlling test points and one for observing them can be combined and the same shift-register can be used for both purposes. This testability method is referred to as Isolated Serial Scan and is illustrated in Fig. 7.14.

For the normal mode of operation, the $NbarT$ input of CUT in Fig. 7.14 is 0. This causes no interference from the test logic except perhaps the delays from the multiplexers. While in this

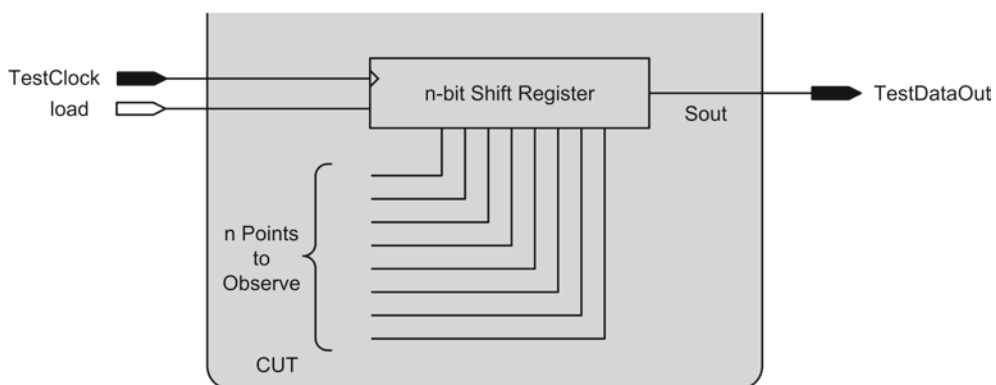


Fig. 7.13 Using a shift-register for simultaneous collection of line values

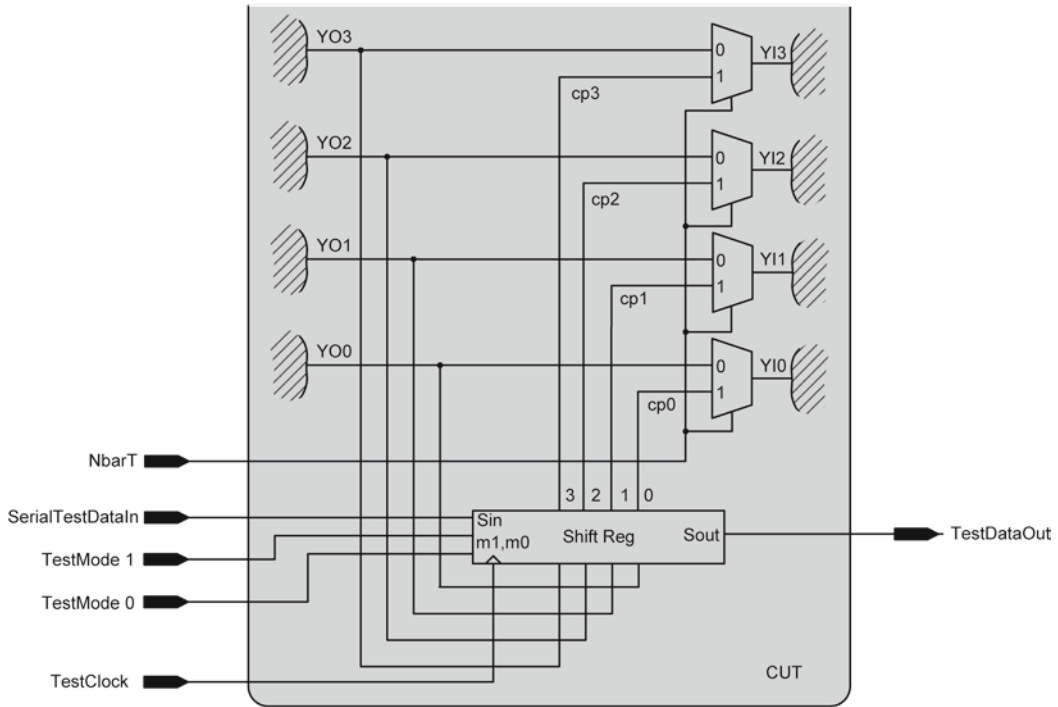


Fig. 7.14 Isolated Serial Scan

mode, the shift-register is put in the serial mode and new test data are shifted in, while collected data from last testing to be observed (from *YO3*, *YO2*, *YO1*, and *YO0*) are being shifted out. The input test data bits that are synchronized with the test clock are placed on the shift-register inputs. At the end of shifting, the CUT is put in the test mode by asserting *NbarT*. This applies the test data in the shift-registers (through the multiplexers) to the inputs of the part of CUT being tested. While this is happening, and the part of CUT is propagating this test data, the shift-register mode is changed to parallel loading. Enough time is given for test data to propagate and shown its affect on *YO* internal output lines. At this time, the shift-register is clocked just once to collect the output. This is followed by the serial mode that shifts in a new test data while shifting out the data that were just collected.

Figure 7.15 shows the Verilog code of the shift-register used in Fig. 7.14. Together, the shift-register and the pseudo code of Fig. 7.16 manage to implement the operation discussed in the above paragraph.

The shift-register shown in Fig. 7.15 performs right shifting when its *TestMode* input is 1 and does the parallel load operation when *TestMode* is 2. *TestMode* = 0 does nothing, and 3 resets the shift-register. The serial output of the shift-register is the same as bit 0 of its parallel output.

Figure 7.16 shows pseudo code for the equipment that connects to the pins of the CUT in Fig. 7.14, and using the DFT hardware that has been placed in the CUT, tests this circuit. Since this code plays the role of an ATE or a device that is testing the CUT, we refer to it as a virtual tester. The code shown works hand-in-hand with the shift-register and other test hardware in CUT.

For testing, *NbarT* is set to 0 and input test data are shifted in while *TestMode* = 1 (shift mode). At the end of shifting, *TestMode* is set to 2 (parallel load), and output test data are captured. This **while** loop repeats for as long as CUT is being tested. Each time a new test data are shifted in.

```

module shiftreg # (parameter size = 4) (input [size-1:0] parin, input
    TestClock, SerialTestDataIn,
    TestClock, input [1:0] TestMode,
    output TestDataOut,
    output reg [size-1:0] parout);

    always @ (posedge TestClock) begin
        case (TestMode)
            0: parout <= parout;
            1: parout <= {SerialTestDataIn, parout[size-1:1]};
            2: parout <= parin;
            3: parout <= 0;
        endcase
    end
    assign SerialTestDataOut = parout[0];
endmodule

```

Fig. 7.15 Isolated Serial Scan shift-register

```

Virtual Tester:Isolated Serial Scan
While (testing) begin
     $\bar{N}/T = 0$ ;
    For (i=1; i<= numbTestPoints; i++) begin
         $\bar{N}/T = 0$ ; TestMode = 1
        Place serial test data on SerialTestDataIn;
        Clock shift-register (TestClock)
        Collect TestDataOut;
    End for
     $\bar{N}/T = 1$ ; TestMode =2;
    Clock shift-register (TestClock)
End
End Isolated Serial Scan

```

Fig. 7.16 Isolated Serial Scan virtual tester

Of all the hardware structures we discussed above, Isolated Serial Scan was the most complete and most complex to operate. This structure provides a background for standard scan architectures that are discussed next.

7.3 Full Scan DFT Technique

With the background provided in Sect. 7.2, we are now ready to discuss standard practices in DFT. The most common DFT technique is full scan that we discuss here [5]. This technique can be explained as one that is very similar in concept to the isolated scan of the previous section.

The isolated scan has an overhead of a shift-register that has to be inserted in the circuit under test. Inclusion of this register still does not solve the problem of test generation for sequential circuits, and we still have to treat the CUT as a sequential circuit with a few extra test points for adding controllability and observability. Full scan takes care of the hardware overhead and sequentiality of CUT by incorporating the required shift-register in the CUT's state flip-flops. As we will see, this reduces the hardware overhead, and at the same time CUT becomes virtually a combinational circuit.

7.3.1 Full Scan Insertion

For scan insertion, we start with the Huffman model of a sequential circuit as shown in Fig. 7.1. As discussed in Sect. 7.1, unfolding a sequential circuit (Fig. 7.2) and applying our testing to the combinational part of a circuit covers all logic and register interconnection faults. Full scan testing takes the Huffman model of Fig. 7.1, and by inserting a shift-register in its register structures, makes a virtual model of the unfolded circuit of Fig. 7.2.

7.3.1.1 Scan Register

Figure 7.17 shows scan insertion in the sequential circuit model of Fig. 7.1. The register part of the Huffman model now includes a shift mode that serially loads *STD*I (Serial Test Data In) into the shift-register. In the normal mode, the register in Fig. 7.17 performs parallel loading.

Figure 7.18 shows Verilog code for the register of Fig. 7.17. The general operation of this register is similar to the one used for isolated scan, the Verilog code of which was shown in Fig. 7.15. Only the signals for control of serial and parallel modes are different.

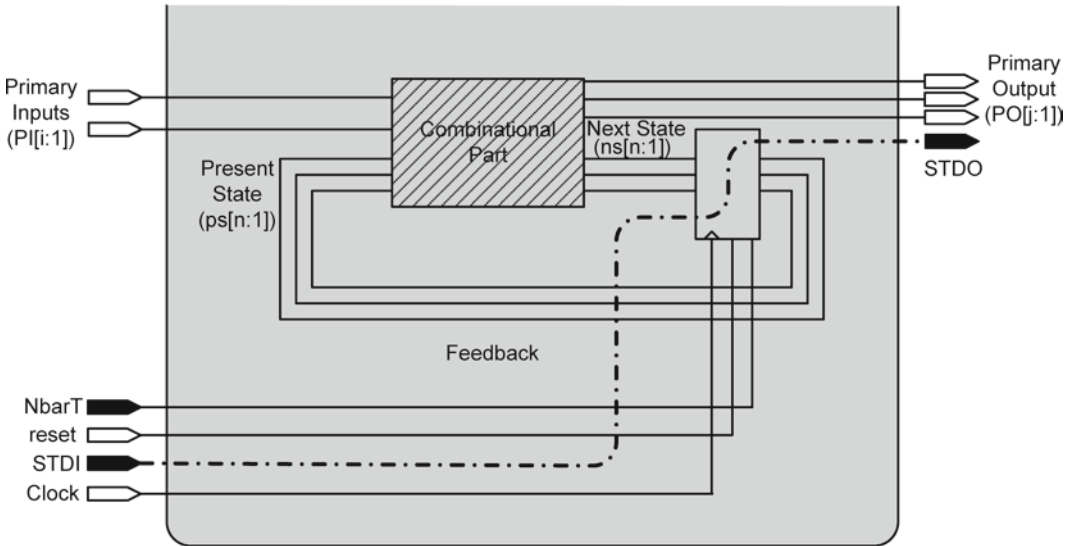


Fig. 7.17 Scan insertion

```

module FBRegister #(parameter size = 4) (input [size-1:0] ns, input
                                         STD I, Clock, NbarT, Reset, output STD O,
                                         output reg [size-1:0] ps);

    always @ (posedge Clock) begin
        if (Reset == 1'b0) begin
            if (NbarT == 1'b0) ps <= ns;
            else ps <= {STD I, ps[size-1:1] }; //shift mode
        end
        else
            ps <= 0;
        end
        assign STD O = ps[0];
    endmodule

```

Fig. 7.18 Testable Huffman model feedback register

The feedback shift-register in Fig. 7.18 works in normal mode by loading ns (next state) into ps (present state) when $NbarT$ is 0. In test mode ($NbarT = 1$), the register becomes a shift-register with $STDI$ serial input and $STDO$ serial output.

Here, we have used a simple signaling for the control of the shift-register, in which $NbarT$ is used for both normal mode (when 0) and shift mode (when 1). The exact signaling depends on the type of the flip-flops used and their data selection mechanisms. These issues will be discussed in Sect. 7.3.2.

7.3.1.2 Test Procedure

The test procedure for the testable Huffman model of Fig. 7.17 only involves testing the combinational part, and the modified register is used as a mechanism for providing controllability of ps and observability of ns . The test architecture shown makes ps inputs the pseudo primary inputs and ns the pseudo primary outputs of the combinational part.

In the normal mode ($NbarT = 0$), the register loads ns into ps . In the test mode, $NbarT$ becomes 1 and test data that is to be applied to ps input of the combinational logic is shifted into the register. When all data bits are shifted, the first part of test data becomes ready at the circuit's ps input, which is its PPI in test mode. At this time, the second part of the test data will be applied to the circuit's primary inputs through external pins. The combinational circuit in Huffman model takes the two-part test data (PI and PPI) and propagates it to its outputs. The primary outputs of the circuit will be available on circuit pins immediately. This is collected and stored.

We then put the circuit in the normal mode of operation by setting $NbarT$ to 0, and clocking the circuit only once. This causes the ns part of the output of the combinational circuit, which is its PPO, to get clocked into the register (in parallel). This PPO will now be shifted out, so that together with the circuit's PO forms the complete output of the combinational part. While this shifting is happening, we will also shift in a new test data into the shift-register for the next round of testing.

Timing and implementation of this testing process depends on the type of flip-flops and the corresponding selection logic structures that we use. The next section discusses this.

7.3.2 Flip-flop Structures

The Verilog code of Fig. 7.18 assumes a simple flip-flop structure and a multiplexer for selection of normal and shift modes. In addition to this structure, there are other structures that are more efficient in terms of timing and gate structures, which we will discuss here.

7.3.2.1 Latches and Flip-flop

A static clocked latch is formed by a cross-coupled gate structure and gates for implementing a clocking mechanism. Figure 7.19 shows an SR-latch, its equivalent NAND structure, and a corresponding symbol.

The cross-coupled structures provide the memory, and the other two gates handle the clocking. The symbol shown indicates dependency of S and R on the clock. Latches are transparent, meaning that when clock is active, S and R inputs directly affect the Q output. Unless complemented by other logic structures or other latches, transparent latches cannot be used in feedback paths in sequential circuits (e.g., Fig. 7.1).

A D-type latch and its corresponding symbol are shown in Fig. 7.20. This is also a transparent latch, and the D input drives the Q output while C is 1.

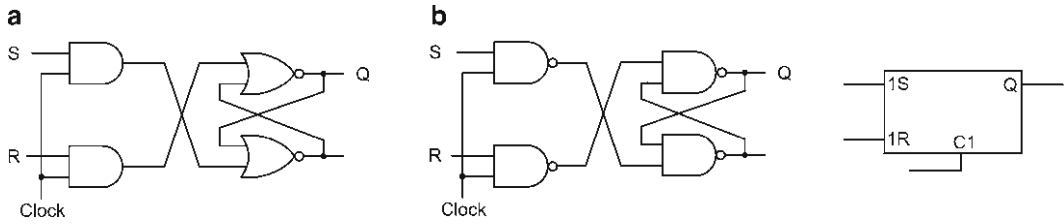


Fig. 7.19 Basic latch

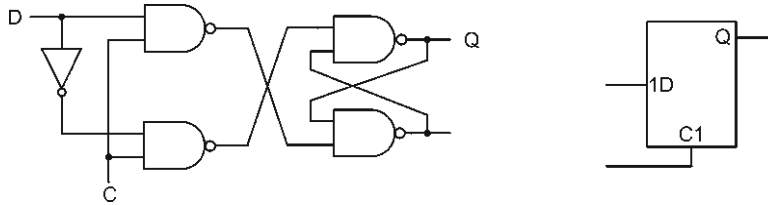


Fig. 7.20 D-latch

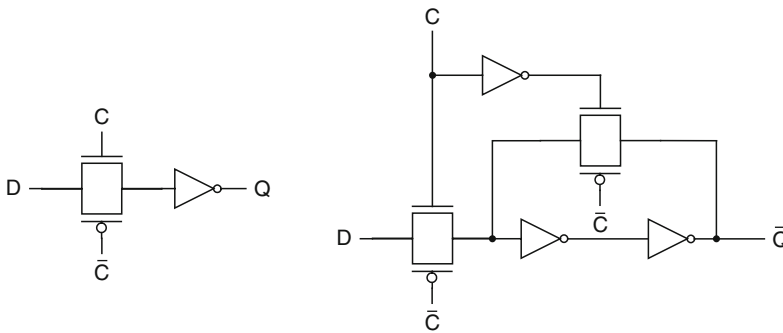


Fig. 7.21 CMOS latches

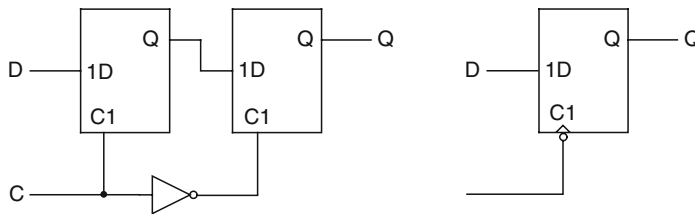


Fig. 7.22 D-type flip-flop

In CMOS, dynamic or pseudo static latches can be built with fewer gates and transistors. Figure 7.21 shows dynamic and pseudo static latches. Test flip-flop architectures that we present here and in the following discussions, that are based on the static latches, also apply to dynamic and pseudo static latches with minor changes.

A flip-flop is built by using two latches with complementary clocks. Flip-flops are not transparent and can be used in sequential circuit feedback paths. Instead of using the complement of the flip-flop clocks (as shown in Fig. 7.22), CMOS uses two nonoverlapping clock phases. Figure 7.22 shows a D-type flip-flop and its symbol. The output of the flip-flop receives the input D after the

falling edge of the clock. Although the symbol shown is for triggering on the clock edge, rather than after the clock edge, this symbol is often used for both flip-flop timings. The flip-flop structure shown here is often referred to as a master-slave flip-flop, where the left latch is referred to as master and the right one as slave.

7.3.2.2 Multiplexed Test Data

The flip-flop of Fig. 7.23 provides a close correspondence with the Verilog description of Fig. 7.18. This flip-flop can be used as a scan element for the feedback register in Fig. 7.17. Figure 7.23 also shows gate-level details of the multiplexer with enable input that we use for flip-flop reset.

The flip-flop shown here has a multiplexer that uses $NbarT$ to select between $DataIn$ (when 0) and $SerialIn$ (when 1). The multiplexer active low enable input provides a synchronous active high reset input. Figure 7.24 shows how three such flip-flops are used as a scan register for the testable model of Fig. 7.17. As before, the hatched areas in this diagram represent the combinational part of the circuit, here that of Huffman model.

In normal mode of operation, when $NbarT$ is 0, $ns[2:0]$ loads into $ps[2:0]$ on the falling edge of *Clock*. In the test mode when $NbarT$ is 1, serial data bits on *STDI* clock into the feedback register in three consecutive clocks, after which time they become available on $ps[2:0]$. The right-most bit is the serial output (*STDO*).

The Verilog code for the flip-flop structure of Fig. 7.23, including the multiplexer, is shown in Fig. 7.25. For modeling multi-bit feedback registers, we can use a Verilog code similar to the scan code in Fig. 7.18, or we can individually cascade flip-flops of Fig. 7.25 to form the right size register. For a behavioral description of a scan-inserted circuit, Verilog code of Fig. 7.18 may be more appropriate, while in a netlist where low-level detailed simulations may be needed, forming the feedback register by cascading individual flip-flops may be more useful.

The flip-flop discussed here is simple, but has the problem of multiplexer delay that adds to the logic delay. In fact, this structure increases the worst-case delay of the circuit for which scan is inserted. This reduces the speed of the normal system clock, and thus, a slower overall operation.

7.3.2.3 Dual Clocking

As shown in Fig. 7.17, with each normal mode clock, the ps output of the feedback register, that is the input of the combinational circuit, must travel through the entire combinational part to affect this part's ns output [6]. This involves a delay, only after which the register can be clocked again.

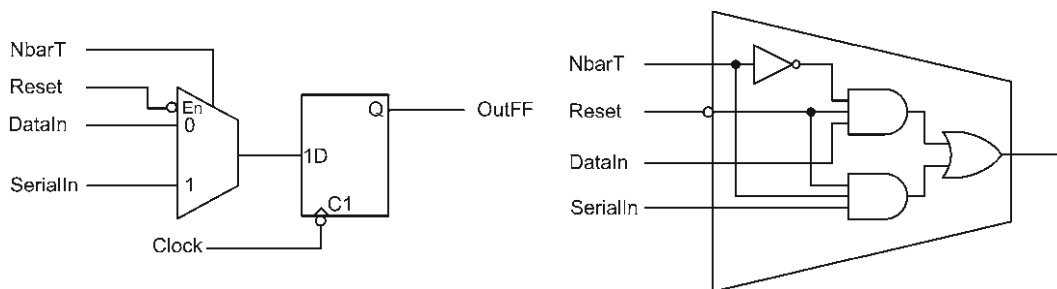


Fig. 7.23 Multiplexed scan element

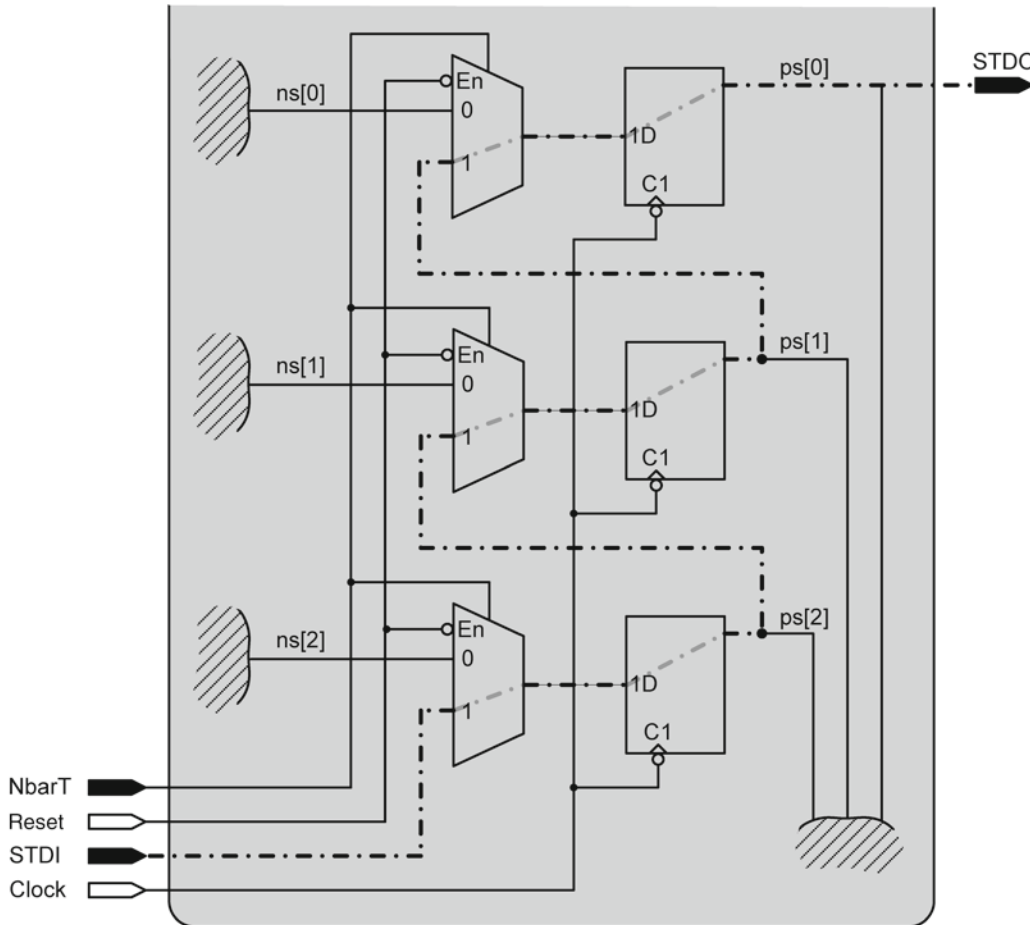


Fig. 7.24 Scan register with multiplexed flip-flops

```

module MuxedFF (input NbarT, Reset, DataIn, SerialIn, Clock, output reg OutFF);
  always @ (negedge Clock) begin
    if (Reset) OutFF <= 1'b0;
    else OutFF <= ~NbarT ? DataIn : SerialIn;
  end
endmodule

```

Fig. 7.25 Multiplexed scan element Verilog code

This delay is the worst-case delay of the CUT, and its normal clock speed has to be slow enough to allow the complete propagation of ps into ns through the combinational part.

On the other hand, such a delay does not necessarily apply when running the circuit in the test mode. In this mode, we are only shifting serial data into the shift-register, and the only logic is that between flip-flop bits (see Fig. 7.24). Therefore, the test mode clocking can be faster than the normal data clocking. In large circuits with many serial bits to shift-in, using a faster clock for the test time gains a good saving in time.

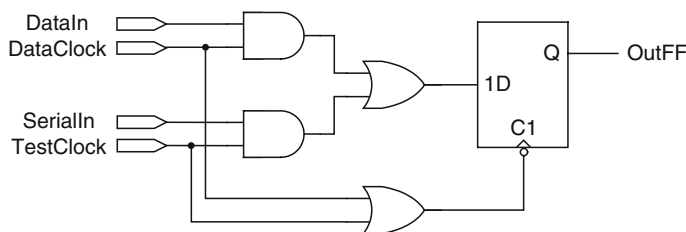


Fig. 7.26 Scan flip-flop with dual clocking

```

module DualClockFF (input DataIn, DataClock, SerialIn, TestClock, output reg OutFF);
    wire Clock;
    assign Clock = DataClock | TestClock;
    always @ (negedge Clock) begin
        if (DataClock) OutFF <= DataIn;
        else if (TestClock) OutFF <= SerialIn;
    end
endmodule

```

Fig. 7.27 Dual clock scan flip-flop Verilog code

The flip-flop of Fig. 7.26 uses dual clocks: one for normal and one for the test mode. The OR gate at the flip-flop clock input causes it to be clocked with either *DataClock* or *TestClock*. The AND–OR logic at the flip-flop input selects *DataIn* when *DataClock* is 1 and selects *SerialIn* when *TestClock* is 1. While either clock is 1, the proper data appear at the flip-flop D-input, and after the clock becomes 0, the data at D are clocked into the flip-flop.

Figure 7.27 shows the Verilog code of the scan flip-flop with the dual clocking system. In this code, an internal clock signal (*Clock*) is made by ORing the two flip-flop clocks. In the **always** statement, that is sensitive to this clock, the data and test clocks are used for conditioning what gets clocked into the flip-flop. Because we have used this procedure and not the exact gate-level equivalent of circuit of Fig. 7.27, our Verilog model does not represent the timing details of the actual circuit.

The problem with this clocking scheme is the hazard that may occur in the logic at the flip-flop D-input. In addition, the problem of introducing the logic gates at the inputs of the circuit flip-flops and increasing the worst-case delay of the circuit still remains.

7.3.2.4 Two-port Flip-flops

To reduce the flip-flop D-input logic delay, the clocking scheme of Fig. 7.28 can be used. The figure shows gate-level details of the flip-flop complementary clock latches. The logic of the right-hand side latch (slave) remains the same as that of Fig. 7.19, and the other latch (master), combines its clocking logic with its required selection logic. To make explanation of the logical operation of the flip-flop in Fig. 7.28 easier, we use the AND–NOR logic of Fig. 7.19 instead of its NAND equivalent shown in the same figure.

In a timing diagram, Fig. 7.29 shows the operation of the two-port flip-flop of Fig. 7.28. For loading *DataIn* into the flip-flop, *ClockA* and *ClockB* are asserted alternatively, while *ClockC* remains at 0. For loading *SerialIn*, *ClockC* and *ClockB* are applied in alternative orders, and in this case *ClockA* is inactive.

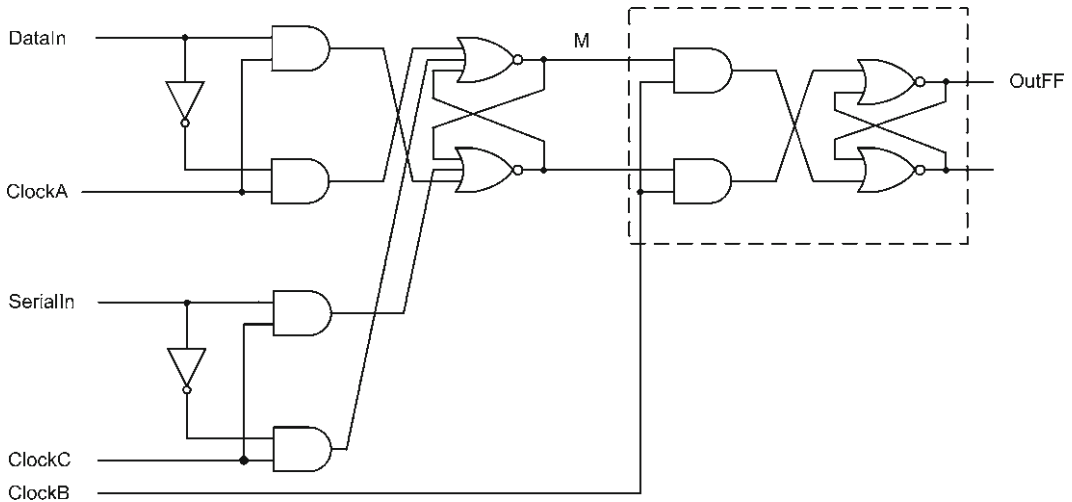


Fig. 7.28 Two-port three-clock flip-flop

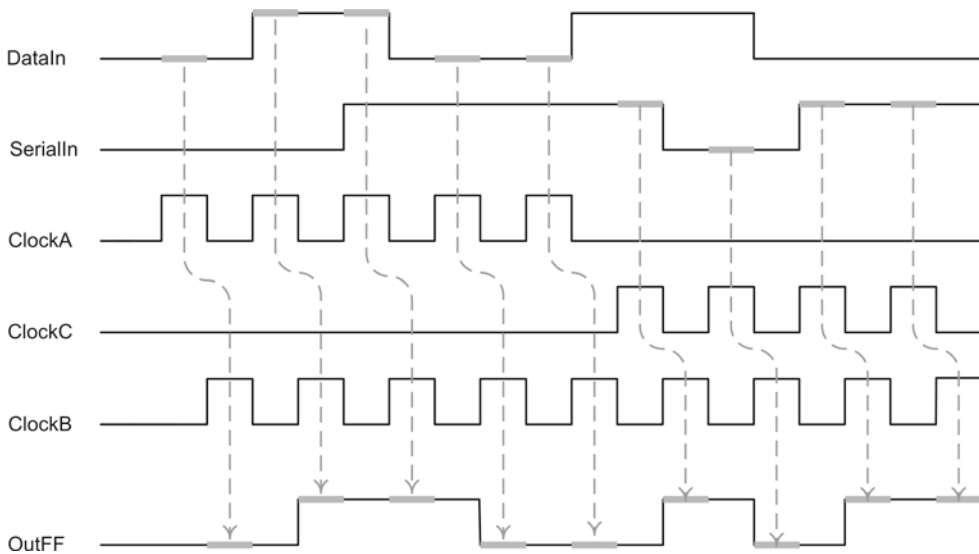


Fig. 7.29 Two-port flip-flop timing

When *ClockA* is asserted, while *ClockB* is 0, data on *DataIn* is latched into the master latch and appears on *M*. When *ClockB* is asserted, data on *M* is latched into the flip-flop output. The situation is similar, when *ClockA* is inactive and *ClockB* toggles.

Figure 7.30 shows a symbol for the structure of Fig. 7.28 that is based on two latches. The master latch is a two-port latch and the slave is a standard D-latch such as that of Fig. 7.20.

Figure 7.31 shows the register part of Fig. 7.17 implemented with the flip-flop of Fig. 7.30. Although shift and normal mode signals are different than the multiplexed flip-flop design of Fig. 7.24, the overall operation remains the same. For normal operation, *ClockA* and *ClockB* are

Fig. 7.30 Two-port flip-flop symbol

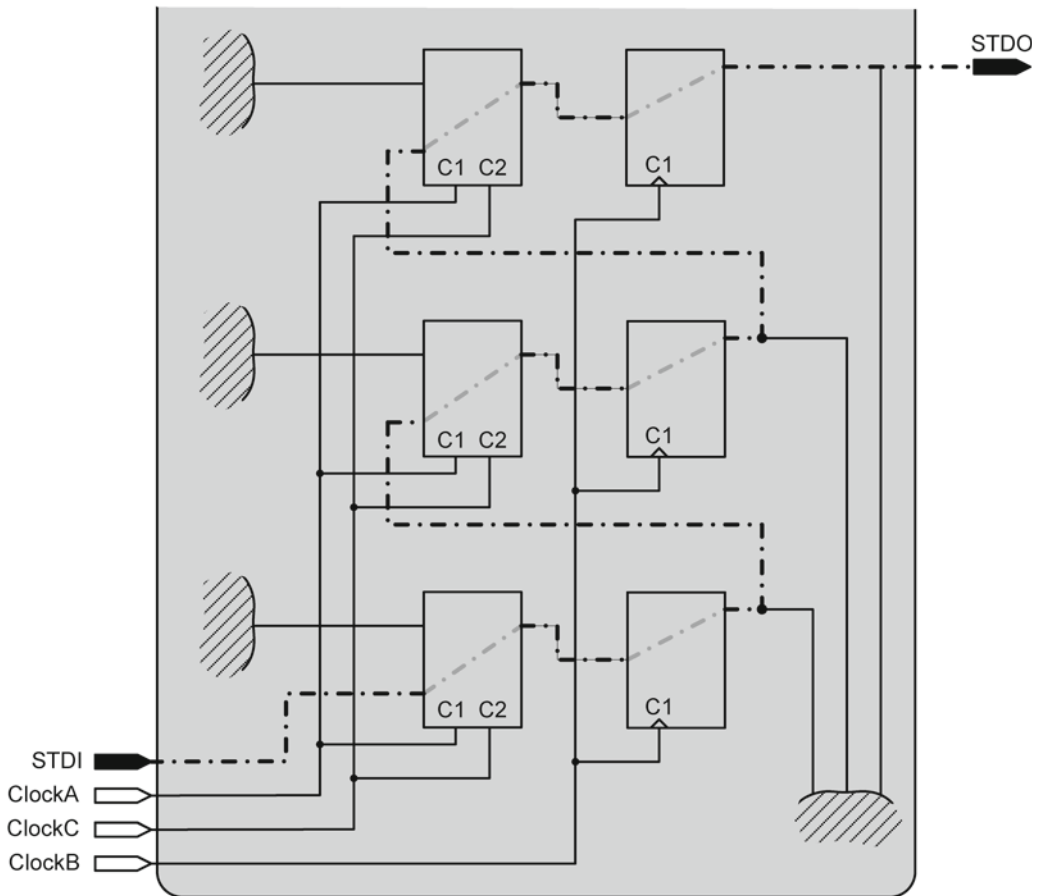
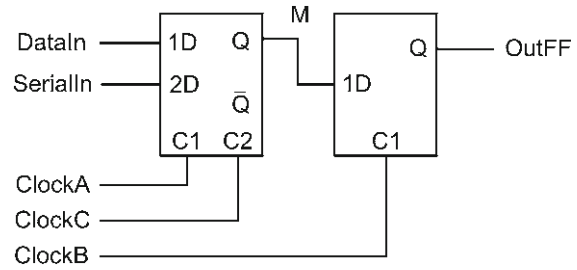


Fig. 7.31 LSSD Design

used, and in test mode, *ClockC* and *ClockB* become complementary clocks. The design shown here is called LSSD (Level Sensitive Scan Design) and was first used by IBM in 1977. Scan path for shifting serial test data into feedback registers is highlighted in this figure.

Figure 7.32 shows Verilog code of the flip-flop discussed above. Signal names apply to those shown in Fig. 7.30. This code is based on description of two latches. The master latch uses two clocks, and slave uses *ClockB*. The output is changed when *ClockB* becomes 1.

```

module DualPortFF (input DataIn, SerialIn, ClockA, ClockB, ClockC, output reg OutFF);
    reg M;
    always @ (DataIn, SerialIn, ClockA, ClockC) begin
        if (ClockA) M <= DataIn;
        else if (ClockC) M <= SerialIn;
    end
    always @ (M, ClockB)
        if (ClockB) OutFF <= M;
endmodule

```

Fig. 7.32 Dual port flip-flop Verilog code

The two-port 3-clock flip-flop of Fig. 7.28 or Fig. 7.30 avoids the multiplexer delay of the previously mentioned flip-flops. The clocking mechanism allows different clock speeds for normal and test modes. This structure has the overhead of having to handle three clock signals.

7.3.3 Full Scan Design and Test

So far in Sect. 7.3, we have talked about scan hardware, scan operation, and testing a circuit that has become testable with this full scan DFT technique. This section shows the complete flow of DFT from a problem specification to generating test and developing a test program.

We show how test methods of the previous chapters fit together with the full scan DFT techniques. Some of the topics discussed earlier in this section such as the full scan test procedure will be elaborated on, and the timing details will be explained in Verilog procedures.

The example that is used is the Residue-5 circuit that was first presented in Chap. 2. This is a sequential circuit, for the testing of which DFT techniques are essential. Figure 7.33 shows the behavioral description of this design. In the discussion that follows, this design will be taken through the following steps:

- Design and validation and design
- Synthesis and netlist generation
- Unfolding
- Combinational test generation
- Scan insertion
- Developing a virtual tester
- Test set verification

7.3.3.1 Design and Design Validation

As mentioned, our example design is the Residue-5 circuit discussed in this and several other chapters in this book. This hardware is described in Verilog (Fig. 7.33). The coding style is according to the Huffman model of Fig. 7.1. The register part specifies an asynchronous resetting mechanism, which means that the reset signal does not participate in the combinational part of the circuit. Keeping reset and other flip-flop control signals away from the combinational part is good for postmanufacturing testing.

The design described in an HDL must be validated. For this purpose, a testbench for functional testing of the design must be developed. This topic has been covered in Chap. 2 and will not be repeated here.

Although we have not taken advantage of this opportunity, the combined design and test environment that we are presenting here allows a testbench for design validation to be used as a template

```

module residue5(input clk, reset, input[1:0] in, output[2:0] out);
reg[2:0] nState, pState;
parameter zero = 3'b000, one = 3'b001, two = 3'b010, three = 3'b011,
           four = 3'b100;

always@(posedge clk, posedge reset)
    if(reset) pState = zero;
    else pState = nState;

always@(pState, in) begin

    case(pState)
    zero:
        case(in)
            2'b00: nState = zero;
            2'b01: nState = one;
            2'b10: nState = two;
            2'b11: nState = three;
        endcase

    one:
        case(in)
            2'b00: nState = one;
            2'b01: nState = two;
            2'b10: nState = three;
            2'b11: nState = four;
        endcase
    . . .

    four:
        case(in)
            2'b00: nState = four;
            2'b01: nState = zero;
            2'b10: nState = one;
            2'b11: nState = two;
        endcase
    endcase

    end//always
    assign out = pState;
endmodule

```

Fig. 7.33 *residue5* partial Verilog code

for a testbench or a test program for postmanufacturing testing. Starting with a testbench for design validation, and gradually improving it for postsynthesis and eventually for manufacturing test is referred to as “testbench migration”. In testbench migration, a testbench begins with just functional test data for verifying the presynthesis design of a circuit and gradually changes to include test data generated by ATPG programs.

7.3.3.2 Synthesis and Netlist Generation

The next step after design validation is synthesis. Using an FPGA-based synthesis program and a netlist converter, we have successfully synthesized the Verilog code of Fig. 7.33. The result is shown

```

module residue5_net(clk, reset, in, out);
input clk;
input reset;
input [1:0]in;
output [2:0]out;
wire
wire_1,
wire_2,
. . .
in_0_0,
in_0_1,
. . .
out_0_0,
out_0_1,
. . .

pin #(2) pin_0 ({in[0], in[1]}, {in_0, in_1});
pout #(3) pout_0 ({out_0_7, out_1_7, out_2_5}, {out[0], out[1],
out[2]});
fanout_n #(8, 0, 0) FANOUT_3 (in_0, {in_0_0, in_0_1, in_0_2, in_0_3,
in_0_4, in_0_5, in_0_6, in_0_7});
fanout_n #(7, 0, 0) FANOUT_4 (in_1, {in_1_0, in_1_1, in_1_2, in_1_3,
in_1_4, in_1_5, in_1_6});
. . .
notg #(0, 0) NOT_1 (WIRE_3, in_1_0);
notg #(0, 0) NOT_2 (WIRE_4, out_2_0);
. . .
and_n #(3, 0, 0) AND_14 (wire_25, {wire_6_5, wire_3_4, out_2_4});
or_n #(4, 0, 0) OR_2 (wire_21, {wire_25, wire_24, wire_23, wire_22});
dff INS_1 (out_0, wire_1, clk, reset, 1'b0, 1'b1, NbarT, Si, 1'b0);
dff INS_2 (out_1,wire_13, clk, reset, 1'b0, 1'b1, NbarT, Si, 1'b0);
dff INS_3 (out_2,wire_21, clk, reset, 1'b0, 1'b1, NbarT, Si, 1'b0);

endmodule

```

Fig. 7.34 Postsynthesis *residue5* netlist

in Fig. 7.34. This netlist uses primitives that are compatible with our utility PLI functions for fault collapsing, fault simulation, and test generation. This netlist, the components of which are shown in Appendix B, is automatically generated from the circuit's behavioral description discussed in Appendix F.

As expected, the netlist includes a logical block consisting of basic gates with a feedback through three flip-flops.

This is compatible with the Huffman model of Fig. 7.1. An example for logic feedback through the flip-flops can be seen by tracing wire *wire_13* that goes to the D input of *INS_2* flip-flop.

Before going to the next step of design, it is necessary to perform postsynthesis simulation of this netlist and make sure it is a correct translation of the behavioral model of Fig. 7.33.

7.3.3.3 Unfolding

Once we have verified that our synthesis has been performed correctly, we start the test generation process. The circuit in Fig. 7.34 is a sequential circuit, and test generation methods and programs for sequential circuits are not very efficient in terms of fault coverage. For this reason, we convert our CUT to a combinational circuit by unfolding it, as presented in Fig. 7.2.

Figure 7.35 shows the *residue5* netlist after being unfolded. As discussed, unfolding means removing flip-flops and making their outputs and inputs pseudo primary inputs and pseudo primary

```

module res5_Unfold (*PI,PPI*/{in,out_2, out_1, out_0},
                    /*PO,PPO*/{out,HM_1, HM_13, HM_21});

input [1:0]in;
input out_0, out_1, out_2; //ppIn
output [2:0]out;
output HM_1, HM_13, HM_21; //ppOut
wire
wire_2,
. . .
in_0_0,
in_0_1,
. . .
out_0_0,
out_0_1,
. . .
wire_3_2,
wire_3_3,
. . .
PPI0,
PPI1,
PPI2,
PPO1,
PPO13,
PPO21;

pin # (2) pin_0 ({in[0], in[1]}, {in_0, in_1});
pin # (3) pin_1 ({out_0, out_1, out_2}, {PPI0, PPI1, PPI2}); //Pseudo PI
pout # (3) pout_0 ({out_0_7, out_1_7, out_2_5}, {out[0], out[1],
                out[2]});
pout # (3) pout_1 ({PPO1, PPO13, PPO21}, {wire_1, wire_13, wire_21});
fanout_n # (8, 0, 0) FANOUT_3 (in_0, {in_0_0, in_0_1, in_0_2, in_0_3,
                in_0_4, in_0_5, in_0_6, in_0_7});

. . .
notg # (0, 0) NOT_1 (wire_3, in_1_0);
notg # (0, 0) NOT_2 (wire_4, out_2_0);
. . .
and_n # (3, 0, 0) AND_14 (wire_25, {wire_6_5, wire_3_4, out_2_4});
or_n # (4, 0, 0) OR_2 (PPO21, {wire_25, wire_24, wire_23, wire_22});

//dff INS_1 (out_0, wire_1, clk, reset, 1'b0, 1'b1, NbarT, Si,1'b0);
//dff INS_2 (out_1, wire_13, clk, reset, 1'b0, 1'b1, NbarT, Si,1'b0);
//dff INS_3 (out_2, wire_21, clk, reset, 1'b0, 1'b1, NbarT, Si,1'b0);

endmodule

```

Fig. 7.35 Unfolded *residu5* netlist

outputs. Figure 7.35 shows that *out_0*, *out_1*, and *out_2* that used to be flip-flop outputs are now mapped to *PPI0*, *PPI1*, and *PPI2* that are circuit's pseudo primary input. Similarly, former flip-flop inputs *wire_1*, *wire_13*, and *wire_21* are now mapped to *PPO1*, *PPO13*, and *PPO21* pseudo primary outputs. The new signal mentioned above also appears on circuit port list as inputs and outputs.

7.3.3.4 Combinational TPG

The netlist in Fig. 7.35 represents a combinational circuit and uses our standard primitives. Chapter 5 showed several random TG methods and Verilog testbenches for implementing them. We have

used the AECpt algorithm of Sect. 5.3.2.2 for test generation for this module. The Verilog testbench for this purpose is similar to the code shown in Fig. 5.27 and is not repeated here. The testbench we used generates the collapsed fault list before test generation.

The results from the test generation Verilog program are as follows.

- Number of faults: 104
- Number of test vectors: 26
- Bits per test vector: 5 (2 PI, 3 PPI)
- Fault coverage: 100%

Recall from Chap. 5 that a testbench was developed for sequential test generation of the Residue-5 circuit (Fig. 5.30). That testbench generated 7 test vectors applied over 45 clock cycles (effectively, 45 tests), and resulted in only 82.4% fault coverage. The unfolding of this circuit enabled us to achieve a 100% coverage.

7.3.3.5 Scan Insertion

To facilitate application of tests generated by the procedure discussed above to the actual CUT, a scan for accessing state flip-flop inputs and outputs is inserted in the CUT. With the insertion of this scan, the block diagram of the netlist of *residue5* that was originally modeled as in Fig. 7.1 becomes as that shown in Fig. 7.36. The netlist corresponding to this figure will be shown next.

For this purpose, the postsynthesis netlist of Fig. 7.34 is modified to include the necessary scan flip-flops and signals. The synthesis tool that generated the original netlist used flip-flop types that already included serial shift facilities that were not used in this netlist. Figure 7.37 shows this flip-flop with *NbarT*, serial input (*Si*), and standard control signals. This description corresponds to the flip-flop notation used in Fig. 7.36 for the feedback flip-flop. Note: The block diagram notation for the flip-flops in Fig. 7.36 shows two flip-flop D inputs marked by 1,2D and 1,3D. The notation specifies that both D inputs are controlled by the clock signal number 1. The upper input requires mode 2 to be active and the lower input needs mode 3. Modes 2 and 3 are determined by a 1 or a 0 on the lower-left input of the flip-flop. We will use this notation in flip-flops of this chapter.

The netlist shown in Fig. 7.38 takes advantage of shift features of the flip-flop of Fig. 7.37. This netlist has additional scan control inputs *NbarT* and *Si*.

The *NbarT* input connects to *NbarT* inputs (shift control) of the state flip-flops (*INS_1*, *INS_2*, and *INS_3*, shown in the last part of code of Fig. 7.38). The *Si* input connects to flip-flop 0 (*INS_1*), the output of which goes to the input of the next, eventually forming a chain of three scan flip-flops. Signal *out_2* that is the output of the last flip-flop drives the *So* serial output signal.

7.3.3.6 Developing a Virtual Tester

Figure 7.38 represents a testable circuit. This netlist implements the original desired functionality of the design, as well as our inserted test hardware. Once manufactured, it has to be tested with a test plan that depends on the test architecture that we have developed, i.e., full scan. The test program running on an ATE implements this test plan.

In this section, we use a Verilog testbench to imitate an ATE. The test program running on the ATE that is written in C/C++ or other high-level software languages will be written here in Verilog procedural code. As previously discussed, we refer to this Verilog testbench as a *virtual tester*.

The block diagram of the virtual tester testing the full scan version of the Residue-5 circuit is shown in Fig. 7.39. The main task of the virtual tester is to read predetermined test data from an external file, apply it to the CUT, get the output of the CUT, and compare the response with the

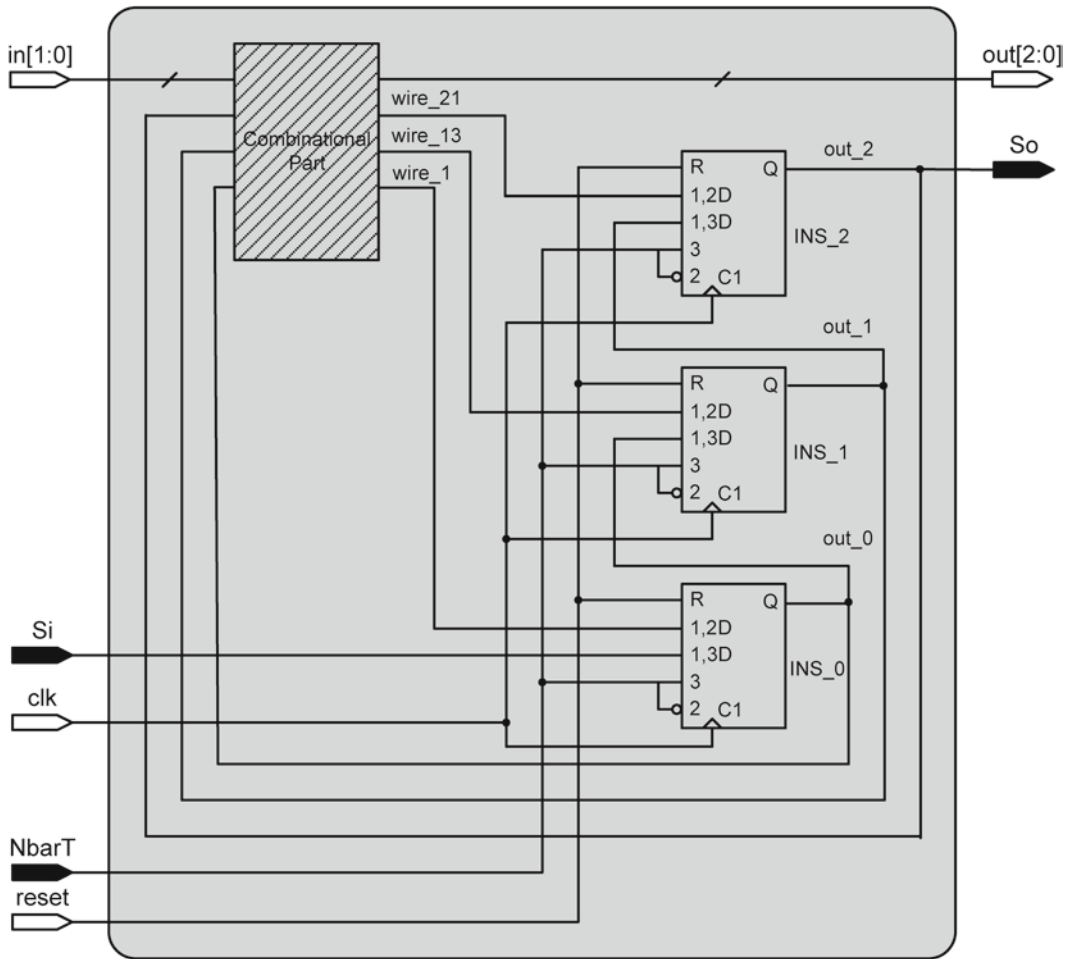


Fig. 7.36 residue5 with scan chain

```

module dff #(parameter tphl = 0, tplh = 0) (Q, D, C, CLR, PRE, CE,
                                         NbarT, Si, global_reset);
input D, C, CLR, PRE, CE, NbarT, Si, global_reset;
output reg Q;
reg val;

always @(posedge C or posedge PRE or posedge CLR) begin
    if(CLR || global_reset)
        val = 1'b0;
    else if(PRE)
        val = 1'b1;
    else if(NbarT)
        val = Si;
    else if(CE)
        val = D;
end

always @(posedge val) #tphl Q = val;
always @(negedge val) #tplh Q = val;
endmodule

```

Fig. 7.37 Flip-flop with scan facilities

```

module res5_ScanInserted(clk, reset, in, out, NbarT, Si, So);
input clk;
input reset;
input [1:0]in;
input NbarT, Si;
output So;
output [2:0]out;
wire
  wire_1,
  wire_2,
  . . .
  in_0_0,
  in_0_1,
  . . .
  out_0_0,
  out_0_1,
  . . .
  pin # (2) pin_0 ({in[0], in[1]}, {in_0, in_1});
  pout # (3) pout_0 ({out_0_7, out_1_7, out_2_5}, {out[0], out[1],
    out[2]});
  fanout_n # (8, 0, 0) FANOUT_3 (in_0, {in_0_0, in_0_1, in_0_2, in_0_3,
    in_0_4, in_0_5, in_0_6, in_0_7});

  . . .
  notg # (0, 0) NOT_1 (wire_3, in_1_0);
  notg # (0, 0) NOT_2 (wire_4, out_2_0);
  xor_n # (2, 0, 0) XOR_2 (wire_13, {out_1_3, wire_14});
  . . .
  and_n # (3, 0, 0) AND_14 (wire_25, {wire_6_5, wire_3_4, out_2_4});
  . . .
  and_n # (3, 0, 0) AND_14 (wire_25, {wire_6_5, wire_3_4, out_2_4});
  or_n # (4, 0, 0) OR_2 (wire_21, {wire_25, wire_24, wire_23, wire_22});

  dff INS_1 (out_0, wire_1, clk, reset, 1'b0, 1'b1, NbarT, Si, 1'b0);
  dff INS_2 (out_1, wire_13, clk, reset, 1'b0, 1'b1, NbarT, out_0, 1'b0);
  dff INS_3 (out_2, wire_21, clk, reset, 1'b0, 1'b1, NbarT, out_1, 1'b0);

assign So = out_2;

endmodule

```

Fig. 7.38 Scan-inserted circuit under test

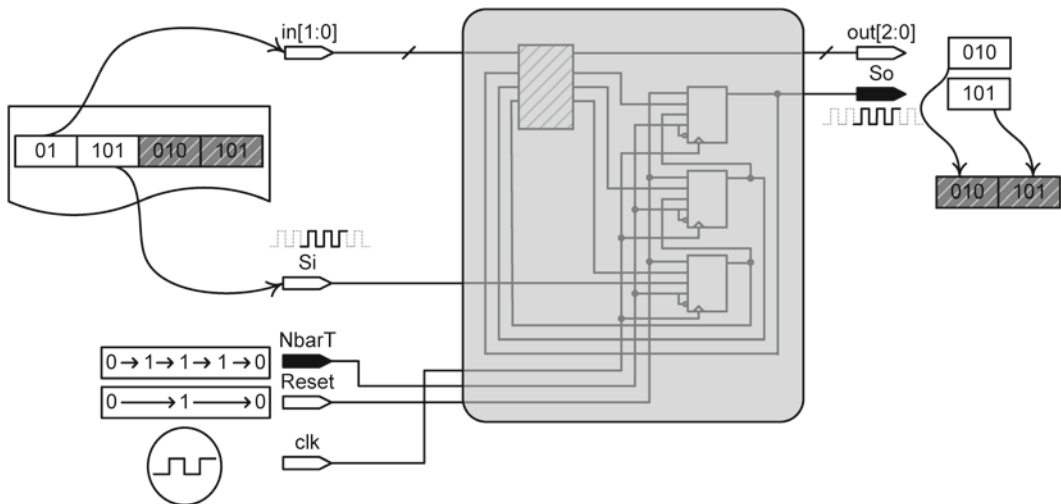


Fig. 7.39 Virtual tester for *residue5*

expected response from the external file. Since we are developing a virtual tester, and our CUT is really not a faulty circuit, our virtual tester also has the responsibility of injecting fictitious faults in the circuit to see if the test set that is provided detects them. We take this process one step further, and inject all circuit faults obtained by fault collapsing in the postsynthesis netlist of Fig. 7.34.

As shown in Fig. 7.39, the parallel data read from the external test file have two parts: one is directly applied to *in[1:0]* input of CUT and the other is serialized and applied through *Si*. The timing of these data is such that when all serial bits have been shifted in the scan chain, the parallel data must be applied to *in[1:0]*.

As shown, the output also has two parts. The first part becomes available on *out[2:0]* immediately after all inputs (i.e., parallel and serial) have been applied. Then, the state outputs (pseudo outputs) become available after the flip-flops have been clocked and then shifted out through *So*.

The Verilog testbench in Fig. 7.40 implements the test environment shown in the block diagram of Fig. 7.39. This testbench reads external data files and through control of clocking of the flip-flops and timing of inputs and outputs applies this test data to the inputs of our CUT, and collects the corresponding outputs. It then compares output data with expected outputs available in the test data file. The paragraphs that follow discuss the manner in which clock data and test inputs of the scan-inserted Residue-5 are controlled that leads to testing this circuit with input/output data that are obtained from the unfolded combinational model of Residue-5.

```

module Tester;

    parameter nff = 3;
    parameter in_size = 2;
    parameter out_size = 3;
    parameter st_size = 3;
    parameter stIndex = 8; /*in_size + out_size + in_size*/
    parameter line_size = st_size * 2 + in_size + out_size;
    reg [numOfPis+ numOfPOs + 2 * numOfDffs - 1 : 0] line;
    . . .
    res5_ScanInserted FUT(clk, reset, PI, PO, NbarT, si, so);

    always #10 clk = ~clk;

    initial begin

        $FaultCollapsing(Tester.FUT, "Res5.flt");

        while( !$feof(faultFile)) begin
            . . .
            while( (!$feof(testFile)) && (!detected)) begin
                . . .
                end//test
            . . .
            end//fault

            $fclose(faultFile);
            $display("number of faults = %f",numOfFaults );
            $display("number of detected faults = %f", numOfDetected );
            $display("Coverage = %f", numOfDetected * 100.0 / numOfFaults);

            $stop;
        end//initial
    endmodule

```

Fig. 7.40 Virtual tester for full scan Residue-5

The testbench in Fig. 7.40 is a generic tester module, at the top of which parameters specifying the number of circuit inputs, outputs, and flip-flops are specified. Other necessary declarations follow the parameter specifications. Variable *line* is declared such that a line from test data file can be read into this variable. Instantiation of the scan-inserted CUT and generation of a periodic clock are also shown here.

The procedural code of the tester starts with the **initial** statement and, immediately following this statement, performs fault collapsing and opens the fault file for subsequent reading. We then have two **while** loops for fault injection and test data application. Display tasks reporting test results appear at the end of the **initial** block.

Figure 7.41 shows the fault injection (the outer **while** loop). In an actual test program for testing a manufactured part, this loop does not exist. In postmanufacturing test, we are testing for existing faults and not intentionally injecting faults. After every fault injection, the loop in Fig. 7.41 resets the CUT and prepares it for testing. Testing is done in the inner **while** loop shown here.

Figure 7.42 shows the details of the inner loop of the procedural code of Fig. 7.40. This **while** loop is responsible for reading a line of test data from *testFile*, applying the input part of it to the circuit, and checking circuit's response with the output part of the line of test data.

A line read from *testFile* has an input part and an output part. Arrangement of inputs and outputs in *line* is shown in Fig. 7.43. The input part has a part that applies directly to the circuit's primary

```

module Tester;
    . . .
    initial begin
        . . .

        while( !$feof(faultFile)) begin

            testFile = $feof("Res5.tst","r");
            status = $fscanf(faultFile,"%s s@%b\n",wireName, stuckAtVal );

            $InjectFault ( wireName, stuckAtVal );

            global_reset = 1'b1; reset = 1'b1; #1;
            global_reset = 1'b0; reset = 1'b0;

            PI = 0;
            cur_expected_st = 0;
            detected = 1'b0;

            while((!$feof(testFile))&&(!detected)) begin
                . . .
                end//test

            if(detected == 0) $display("NOT DETECTED = %s s@%b", wireName,
            stuckAtVal );

            $RemoveFault(wireName);
            numOfFaults = numOfFaults + 1;
            $fclose(testFile);

        end//fault
    end//initial
endmodule

```

Fig. 7.41 Fault injection in Residue-5

```

module Tester;
    ...
    initial begin
        ...
        while( !$feof(faultFile)) begin
            ...
            while((!$feof(testFile)) && (!detected)) begin
                status = $fscanf(testFile, "%b\n", line);

                pre_expected_st = cur_expected_st;

                expected_PO = line[out_size - 1:0];
                cur_expected_st = line[st_size + out_size - 1 : out_size];
                PI = line[st_size+out_size+in_size-1:st_size+out_size];

                @(posedge clk);

                NbarT = 1'b1;
                index = stIndex;

                repeat(nff) begin
                    si = line[index];
                    saved_st[index - stIndex] = so;
                    @(posedge clk);
                    index = index + 1;
                end

                SampledPO = PO;
                NbarT = 1'b0;
                #5;

                if({pre_expected_st, expected_PO} != {saved_st, SampledPO})
                begin
                    numOfDetected = numOfDetected + 1;
                    detected = 1;
                end

            end
            ...
        end
    end
end//initial
endmodule

```

Fig. 7.42 Test application and response collection

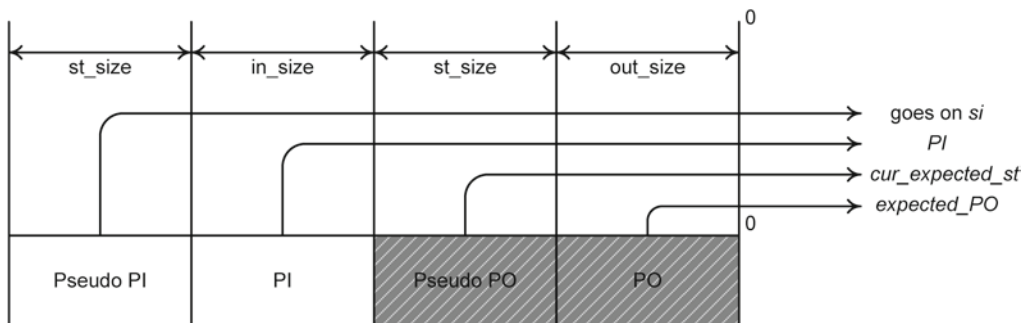


Fig. 7.43 Arrangement of stimulus and response in line

inputs and another part that must be shifted into scan flip-flops (*Pseudo PI*). Similarly, the output part itself has two parts: one part becomes available immediately when proper inputs (*PI*) and scan data are shifted into the flip-flops (*PO*) and a second part that will be shifted out when serial data belonging to the next test vector are being shifted in (*Pseudo PO*). Understanding this timing is crucial in collecting the right outputs and comparing relevant responses.

The *Test* loop in Fig. 7.42 reads stimuli and response data, applies it to the CUT, compares the outputs, and reports if a test detects the injected fault. Data that contain stimuli and response have the format shown in Fig. 7.43. Variable names storing various parts of *line* data are indicated in this figure.

As shown in Fig. 7.42, before *cur_expected_st* (current expected state) is overwritten with new data from *line*, it is saved in *pre_expected_st* (previous expected state). The *pre_expected_st* is saved because the actual state flip-flop contents of the previous test will not become available until current serial data are shifted into the state flip-flops, and that shifted-out data is what needs to be compared with *pre_expected_st*.

The *PI* part of the current data read from *line* is immediately applied to the scan-inserted Residue-5. (See *PI* in port connections of *res5_ScanInserted* in Fig. 7.40.)

The **repeat** loop in Fig. 7.42 takes the pseudo PI part of *line* and shifts it into the circuit's state flip-flops. Meanwhile, flip-flop outputs are serially shifted out and saved in *saved_st*. During the shift operation, the circuit is in test mode ($NbarT = 1$), and each shift is accompanied with the positive edge of clock. Shifting repeats *nff* (number of flip-flops) times.

After shifting completes, with *PI* at the circuit's primary inputs, and pseudo PI part of *line* in the state flip-flops, the circuit produces the output that corresponds to the input data of *line* that was just read. This *PO* is saved in *SampledPO*.

In the last part of the loop in Fig. 7.42, the collected pseudo output and the primary output of the circuit are concatenated and compared with the expected state and the expected primary output.

7.3.3.7 Test Set Verification

The test set for Residue-5 circuit was developed using its unfolded model with no registers, and using combinational test generation methods. We obtained a 100% coverage from this test set. The above procedure, in which this same test set is applied to the actual sequential model of Residue-5, verifies the test set obtained from the combinational model.

7.4 Scan Architectures

This chapter started with ad hoc methods for making designs testable, and gradual improvement of such methods led to the DFT technique that we referred to as full scan. Actually, full scan is part of a larger category of DFT techniques that are generally referred to as scan architectures [5]. This section presents alternatives to a full scan design. We start with repeating the full scan, to set the ground for comparing it with other methods.

As we have said in many instances before in this book, any sequential digital system can be modeled by a combinational circuit with a feedback through a vector of clock-controlled flip-flops. The functional relation between the flip-flops may form one or several clusters of flip-flops that become our feedback registers. In this model, no line is drawn between control and data registers, i.e., feedback registers consist of data and control flip-flops. We use the general diagram of Fig. 7.44 for describing various scan architectures.

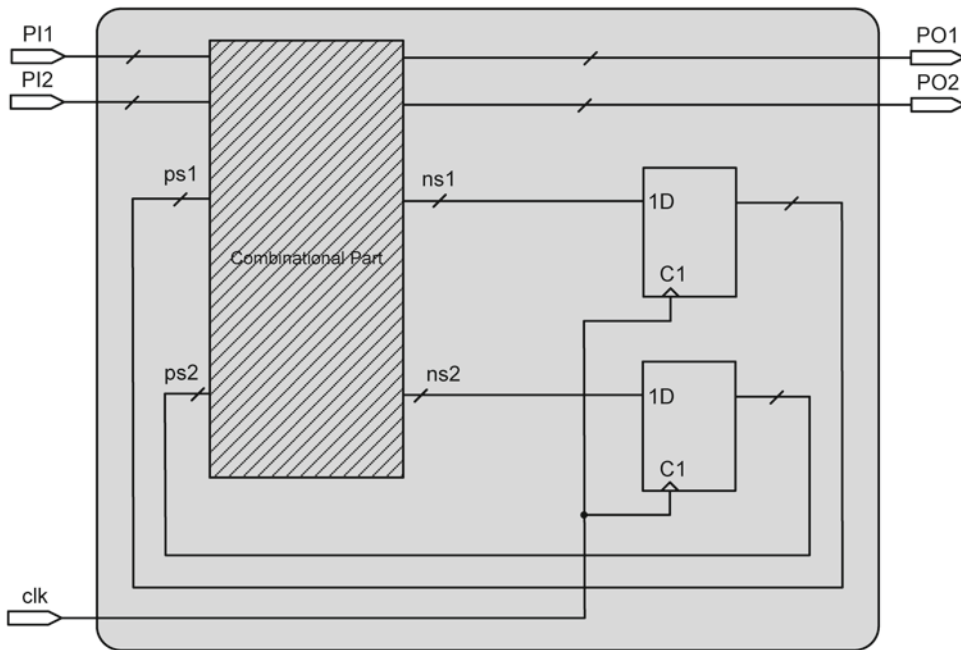


Fig. 7.44 Huffman model with multiple vector inputs, outputs, and states

7.4.1 Full Scan Design

In the full scan DFT techniques, feedback registers are given the additional capability of acting as shift-registers in the test mode. Full scan chains all the registers together and provides a serial-in and a serial-out ports. This method enables serial access for controlling all flip-flop outputs (circuit's present state) and observing all flip-flop inputs (circuit's next state). Full scan refers to the fact that all circuit flip-flops are included in the scan chain. Figure 7.45 shows addition of full scan to the model of Fig. 7.44. The problem with full scan is the long chain of flip-flops that test data have to be shifted into that reflects on the test time [7].

7.4.2 Shadow Register DFT

An alternative design to full scan design is the use of shadow registers. This technique duplicates the feedback registers. It uses one set for normal operation of the circuit and another set for test purposes. This method reduces the test time by overlapping the time of test data preparation and response collection with the normal operation of the circuit [6].

7.4.2.1 Shadow Architecture

Figure 7.46 shows the addition of a shadow register to the model of Fig. 7.44. The new set of registers receives next-state outputs from the combinational part of the circuit and produces outputs that are multiplexed on the present state inputs of the combinational part.

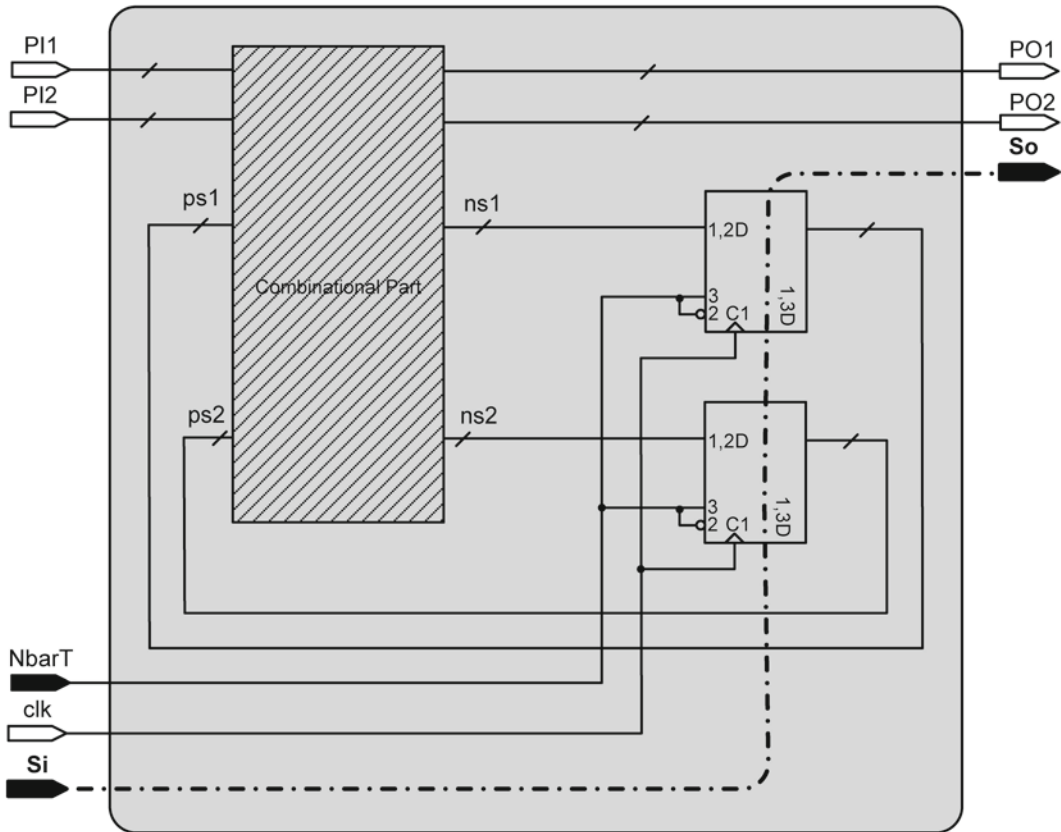


Fig. 7.45 Full scan DFT technique

7.4.2.2 Shadow Test Procedure

In the normal mode ($NbarT = 0$), the circuit has its normal inputs, and normal feedback registers provide data for the present state of the circuit. While in this mode, the shadow registers are put in their serial shift mode. Therefore, simultaneous with the normal operation of the circuit, the test data on Si are shifted into the shadow registers with $Tclk$ test clock. When all test data have been shifted in, $NbarT$ is asserted, which puts the circuit in the test mode and disables shifting serial data into the shadow register. In this mode, the normal circuit clock (clk) is disabled and (see the AND gate on clock) test states ($TS1$ and $TS2$) will drive the circuit's state inputs ($ps1$ and $ps2$).

In the test mode, the $ps1$ and $ps2$ inputs of the combinational part are driven by test data. We will also drive $PI1$ and $PI2$ with their corresponding test data. This will set all inputs of the combinational part of the CUT to previously prepared test data. With test inputs provided to all combinational block inputs, the response becomes available on $PO1$, $PO2$, $ns1$, and $ns2$. The primary output part of the response ($PO1$ and $PO2$) can be read at this time, but $ns1$ and $ns2$ must be clocked into the shadow register and shifted out in order to be observed. Clocking the circuit while $NbarT$ is 1 loads $ns1$ and $ns2$ into the shadow registers (see 2, 3 control inputs of the shadow registers, mode 2 allows serial shift, and mode 3 takes the normal flip-flop input).

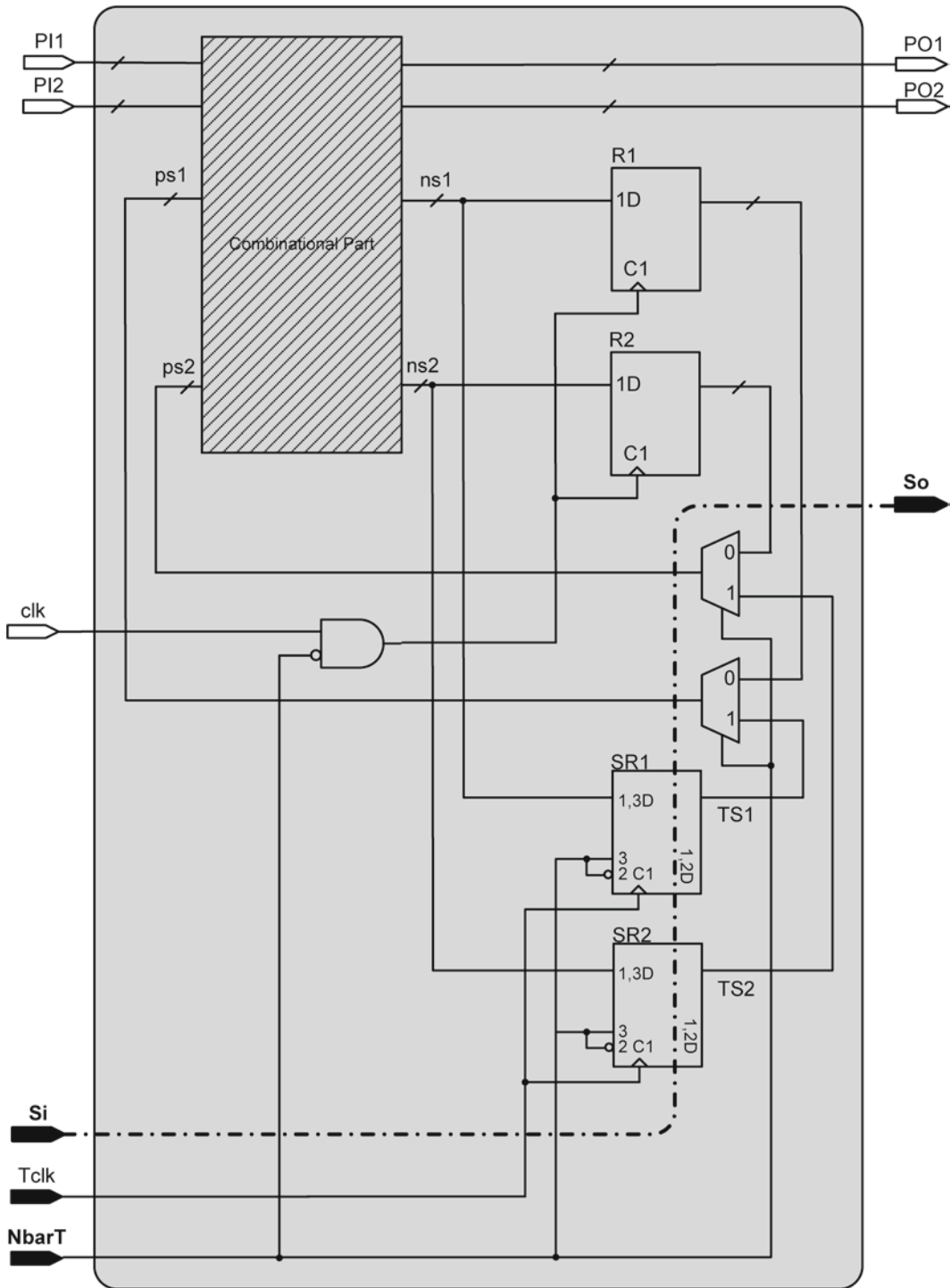


Fig. 7.46 Shadow register

To collect the *ns1* and *ns2* part of the test response, the circuit is put in the normal mode by setting *NbarT* to 0. This puts the shadow registers in the shift mode. In this mode, *clk* is enabled and the circuit goes back to normal mode of operation, new test data inputs start being shifted from *Si*, and the part of test response on *ns1* and *ns2* start being shifted out through *So*.

7.4.2.3 Shadow Versus Full Scan

The biggest advantage of shadow registers is the capability of online testing, and its disadvantage is doubling the number of feedback flip-flops.

Timing wise, having a separate clock for test (*Tclk*) enables faster shifting of test data into the shadow registers. On the other hand, the multiplexers in the feedback path cause a delay in this path which is considered another disadvantage of this technique. This delay slows down the normal clock speed and affects system performance.

For test generation, combinational methods can be used with the unfolded version of the CUT. This is because insertion of shadow registers makes all combinational part inputs and outputs accessible.

7.4.3 Partial Scan Methods

The problem of test time in full scan designs can be alleviated by scan chains that include only part of the feedback registers. As compared with full scan, selecting some of the feedback registers is referred to as partial scan. The method of selecting registers that are put in the scan varies from one partial scan method to another, but in general, selection must be done such that combinational test generation methods can still be used for test data generation [8].

In an RT-level design with a controller and a datapath, partial scan methods generally apply to the datapath. In a datapath, paths through buses and logic leading to registers are searched, looking for registers whose exclusion from a scan chain (inclusion in the CUT test model) would still allow combinational test generation methods to be used for test generation. Below we will discuss one such scenario.

7.4.3.1 A Partial Scan Architecture

Figure 7.47 repeats the circuit model of Fig. 7.44. For presenting our example partial scan method, we assume the combinational part of the circuit we are to add DFT hardware consists of two combinational blocks as shown in the hatched boxes of Fig. 7.47. The circuit primary inputs split and go to both blocks, *A* and *B*, and the outputs of the blocks are merged to form the circuit's primary outputs. One feedback register is driven by block *A* and the other by block *B*. The output of the latter register goes to input of block *B* and that of the former goes to block *A*. As shown in the diagram, the feedbacks are crossed, i.e., *A* through register to *B* and *B* through register to *A*. It is this property that makes partial scan possible in this circuit.

In order to test all the logic in the circuit of Fig. 7.47, it is only necessary to put feedback register *R2* in a scan path. This scan is shown by a heavy dotted line in Fig. 7.47. Applicability of this method to datapath of an RT-level design becomes more clear by turning the diagram of Fig. 7.47 into that of Fig. 7.48.

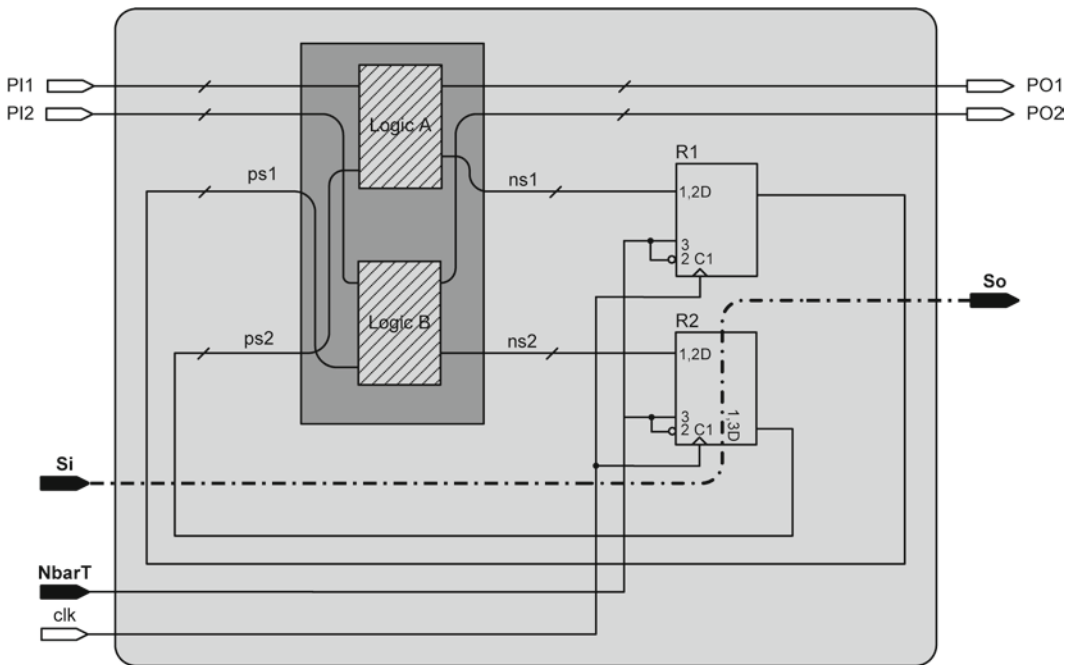


Fig. 7.47 Partial scan, starting with Huffman model

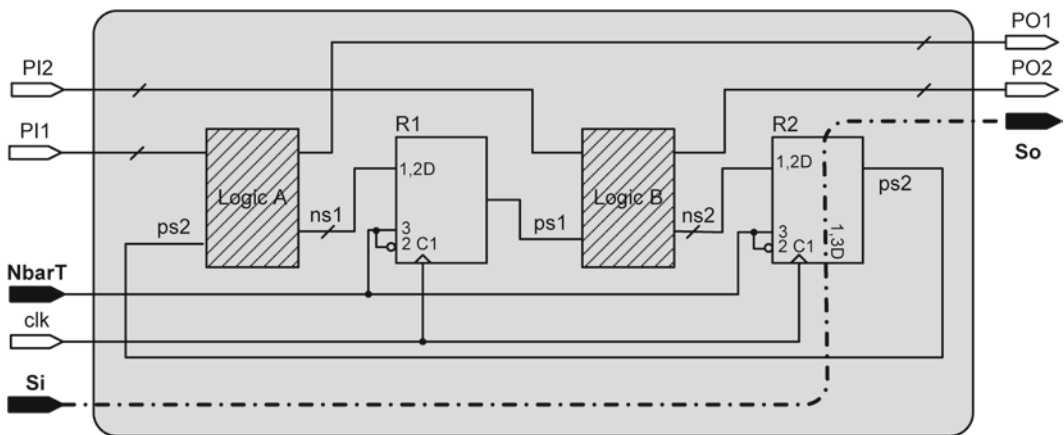


Fig. 7.48 Partial scan datapath

Combinational blocks *A* and *B* can be a mixture of logic units and buses in a datapath. Although it may be difficult to map an entire datapath into an architecture like that of Fig. 7.48, it is not hard to find sections of a large datapath that fit this model. In addition, with minor modifications to a datapath, this and other sub-architectures that qualify for various partial scan designs can be formed.

In the above discussion, we selected a register to be removed from full scan and found that this removal would still qualify the resulting circuit model for combinational test generation. In a large RT-level design, there may be many such choices, some of which may conflict with one

another. Selection of a set of registers to remove from a full scan design is an *np*-complete problem that has been discussed and dealt with in the literature [10].

7.4.3.2 Partial Scan Test Procedure

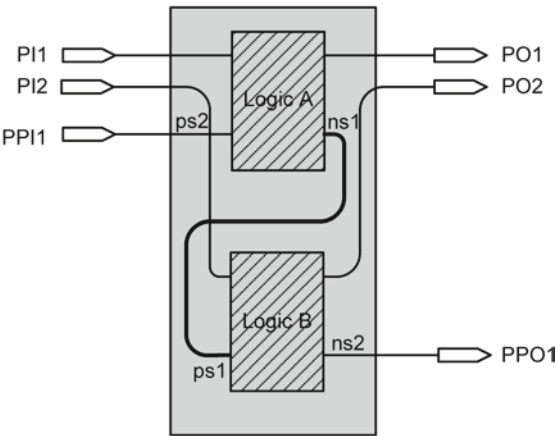
In general, testing a design with partial scan is more involved than one with full scan. A partial scan design requires a sequence of shiftings and register parallel loading to get test data to and out of all logic parts. In spite of this more complex handling of test data, a good partial scan design still uses combinational test generation methods. Testing the design of Fig. 7.47 or Fig. 7.48 is discussed here.

Test generation for the combinational part of circuit of Fig. 7.47 is done by a circuit model that removes *R1*, and then unfolds the circuit by treating input and output of *R2* as pseudo primary output and pseudo primary input, respectively. This combinational model is shown in Fig. 7.49.

As shown, *R1* is removed and its *ns1* input is connected to *ps1* output. Register *R2* unfolds and its input becomes *PPO1* and its output becomes *PPI1* (pseudo output and input). Since this model does not have any feedback loops, it is treated as a combinational circuit, and combinational test generation methods are used for generating test for it. Test vectors generated for this circuit have the form shown in Fig. 7.50. Values for *PPI1*, *PI2*, and *PPI1* inputs are followed by output values *PO1*, *PO2*, and *PPO1*.

What follows discusses application of tests (format of Fig. 7.50) to our partial scan design (Fig. 7.47 or Fig. 7.48), and collection of outputs. Although Figs. 7.47 and 7.48 are equivalent, the discussion below is easier to follow when considering Fig. 7.48.

Fig. 7.49 Partial scan combinational model



PI1	PI2	PPI1	PO1	PO2	PPO1
-----	-----	------	-----	-----	------

Fig. 7.50 Test vector arrangement

In the normal mode of operation, $NbarT$ is 0 and circuit is performing its normal operation. With each clock, $ps1$ receives $ns1$ (register $R1$), and $ps2$ receives $ns2$ (register $R2$). In the test mode, $R1$ is disabled and $R2$ is put in the shift mode. When all data bits belonging to $PP11$ segment of a test vector have been shifted into $R2$ (which means that $ps2$ in Fig. 7.48 has proper test data), the $P11$ segment is applied to the $P11$ input of the circuit. At this time, $PO1$ test response is collected and $ns2$ is generated as well.

We then disable $R2$ and enable $R1$ parallel loading. This is happening while combinational block A (Fig. 7.48) has proper $P11$ and $ps2$ test inputs. Clocking the circuit at this time will load $ns1$ in $R1$, and thus applies it to block B via $ps1$.

The other part of input of block B is $PI2$ that is read from the test vector and applied to this input. Now, block B has all its test data applied to its inputs. The $PO2$ output from block B is read and collected.

Of the three parts of the test vector response, we have two parts ready ($PO1$ and $PO2$). We now put the circuit back in the test mode, and while new test data shift in $R2$, test response of B in $R2$'s $ns2$ input will serially be collected. After the shifting, the three parts of the output ($PO1$, $PO2$, and $PP01$) are ready to be compared with the expected response.

The procedure discussed above is summarized as shown below. This procedure is for the test mode and begins by setting $NbarT$ to 1.

- While $NbarT$ is 1, clock $R2$, shift $PP11$ test data into $R2$, disable $R1$; also of previous test serially collect $PP01$.
- Apply $P11$ test data to $P11$ input, collect response from $PO1$.
- Set $NbarT$ to 0, enable $R1$, clock once.
- Apply $PI2$ test data to $PI2$ input, collect response from $PO2$.
- Keeping $NbarT$ at 0, clock once.
- Return to step 1.

7.4.3.3 Partial Scan Versus Full Scan

Obviously, partial scan reduces test time by having fewer bits to shift in. On the other hand, partial scan has a more complex test procedure as discussed above. The main problem with partial scan is that there is no unique partial scan method, and not all circuits can necessarily take advantage of a partial scan method. For finding proper registers to scan, a topological processing of the circuit is necessary. Configuring a circuit for partial scan must be paralleled with extracting a test procedure that works with the scan design.

A partial scan method such as the one described here fits well with pipeline architectures. In this case, the test procedure becomes dependent on depth of the pipeline.

7.4.4 Multiple Scan Design

The problem of long scan chain can be moderated by using multiple independent or parallel scan chains. In multiple independent scan chains, each scan register has its shift, load, and clock control, whereas in multiple parallel scan chains, all scan registers are controlled by the same set of signals. If registers to be scanned can be put into groups of equal number of cells, then they can be regarded as parallel scan registers with the same set of control signals. However, if the number of flip-flops in the scan chains in a design are not the same, then they need independent shift and clock enable control signals [9].

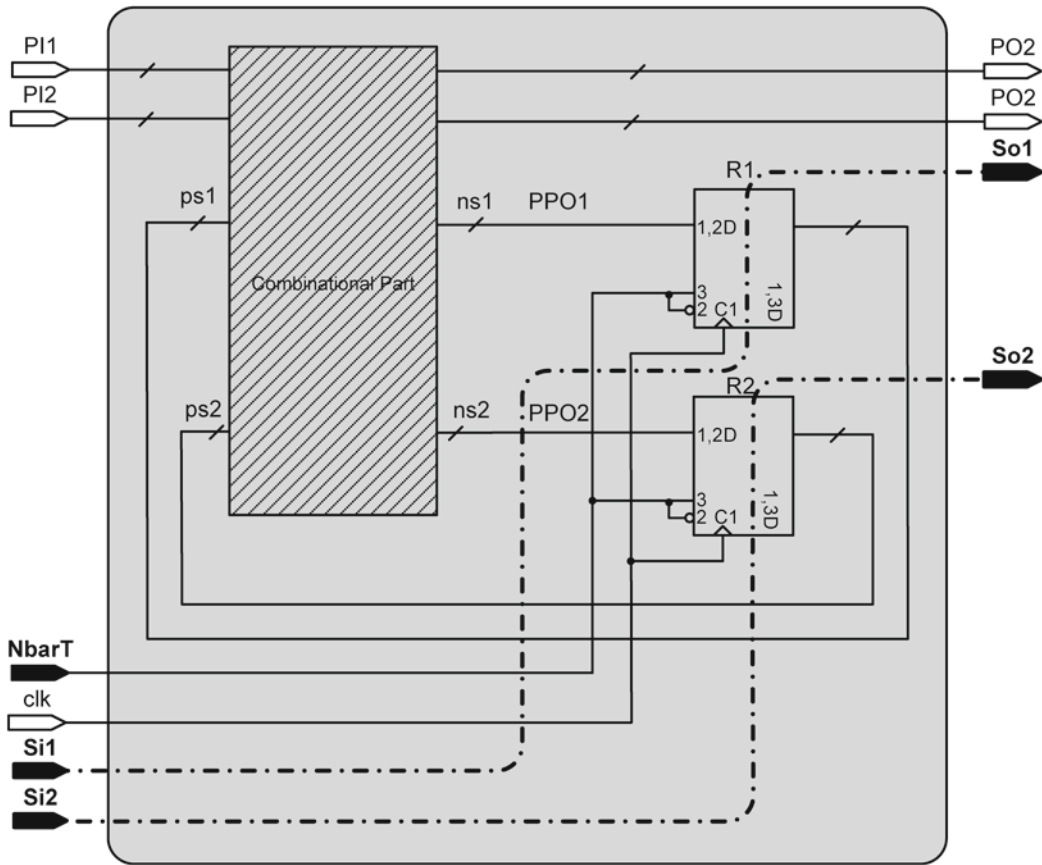


Fig. 7.51 Multiple parallel scan chains

7.4.4.1 Multiple Scan Architecture

Figure 7.51 shows the familiar model that we have been using for demonstrating our scan design. In this figure, $R1$ and $R2$ are put into two separate scan chains with $Si1$ and $Si2$ inputs and $So1$ and $So2$ outputs. We assume here that registers $R1$ and $R2$ have the same length. Therefore, the scan chains have become two parallel registers with the same clock and $NbarT$ control inputs.

7.4.4.2 Multiple Scan Test Procedure

Test generation for a multiple scan design is based on the unfolded model and is no different than test generation for a full scan design. The difference is in application of test vectors through serial test inputs.

Input test data for the circuit of Fig. 7.51 consist of that for $PI1$, $PI2$, $PPI1$, and $PPI2$, and test responses are for $PO1$, $PO2$, $PPO1$, and $PPO2$. For testing this circuit that has two scan chains for $PPI1$ and $PPI2$, individual test data bits from the corresponding test data segments are read and are applied simultaneously to $Si1$ and $Si2$. When $NbarT$ is 1, $R1$ and $R2$ are in the shift mode and shift test data into the registers. While shifting-in is taking place, previous results are collected from $So1$ and $So2$. When shifting is complete, $NbarT$ is set to 0, parallel test data for $PI1$ and $PI2$ are applied to the CUT, and $PO1$ and $PO2$ are read.

7.4.4.3 Compared with Full Scan

Multiple scan significantly reduces test time, with no overhead on test generation procedure. The overhead is on extra test pins, and in case of independent scans, on test clock and normal mode controls.

7.4.5 Other Scan Designs

Depending on the architecture of circuit under test, test data length, test procedure, and other such factors, many variations of scan are possible.

A variation of serial scan is random access scan. In this case, the feedback flip-flops in Fig. 7.44 are put in a memory array. In normal mode of operation, the flip-flops work in parallel mode, and in test mode, individual flip-flops are addressed, read out through scan output, and set to their test values through the scan input. Addressing scan flip-flops in test mode is done by row and column addresses that can be shifted-in, counted to, or a mixture of both.

This method offers fast access to the scan flip-flops, but has a large hardware overhead for implementation of the memory array. Addressing scan flip-flops and extra pins for this purpose are cause of extra hardware and possibly test time reduction.

Often datapath and control parts of a circuit require different DFT methods. For separating control and data parts of a circuit, the isolated scan method can be used. We can then use full scan for the control part, and perhaps partial scan for the data part. Such arrangements are architecture dependent, and too many to enumerate. Hardware designers must be aware of the possibilities and decide on the DFT method to use based on the circumstances. Often, there is no single best solution.

7.5 RT Level Scan Design

The previous section discussed various scan designs for the general sequential circuit model of Fig. 7.44 (Huffman model). In this section, we show how these methods apply to an actual RT-level design with a datapath and a controller.

A complete RT level design can either be partitioned into its datapath and controller, and each part be treated for scan separately, or all the registers and control flip-flops can be lumped together and treated according to the Huffman model of Fig. 7.44. In the example that we present in this section, we take the latter approach and treat all flip-flops similarly. The example that we use is the Adding Machine first presented in Chap. 2.

7.5.1 RTL Design Full Scan

As shown in Fig. 2.30, the Adding Machine has two 8-bit registers for *AC* and *IR*, and it has a 6-bit register for the program counter. With the addition of two control flip-flops, this design has a total of 24 flip-flops. Scan insertion in this design is done as shown in Fig. 7.52. The serial input is the left-most bit of *AC*, and serial output is taken from the least significant control bit.

Although we are showing an RT-level view of the circuit, the actual scan is inserted in the netlist of this circuit. The netlist is obtained by synthesizing the behavioral description of the Adding Machine. The scan is inserted in this netlist manually.

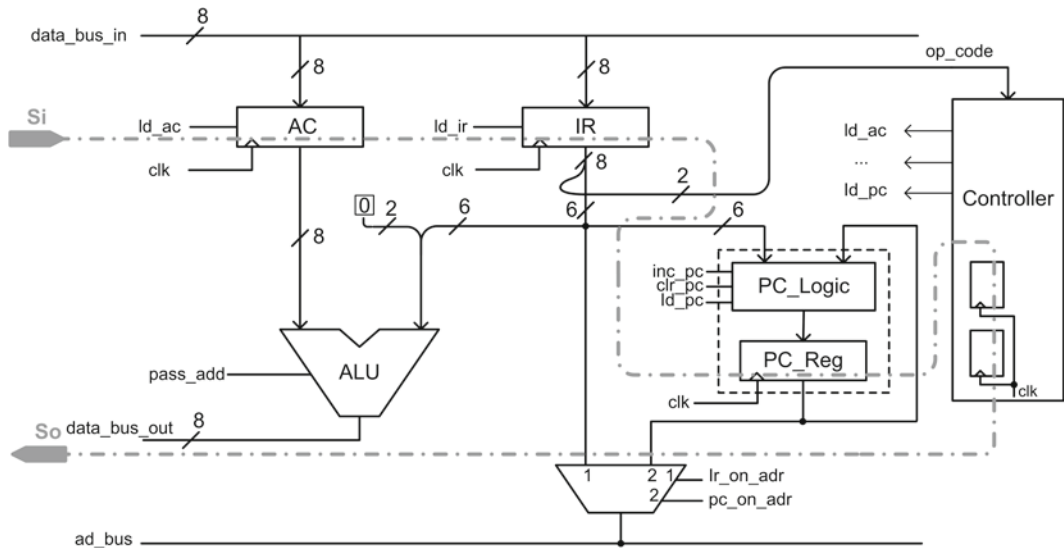


Fig. 7.52 Full scan Adding Machine

With the scan inserted as shown, *data_bus_in* becomes the circuit's primary input, and *data_bus_out* and *ad_bus* together form the primary output. The testable circuit has 24 pseudo primary input and 24 pseudo primary output that are register outputs and inputs, respectively.

For testing this circuit, an unfolded model of the circuit was created, as was done for the Residue-5 example of Sect. 7.3. This combinational netlist was used for test generation by HOPE for 976 faults. HOPE generated 64 tests and obtained coverage of 80.53%. Each test vector has 32 bits on the input side and 38 bits on the output side. From the generated 32 bits, 8 bits are used for *data_bus_in* and the remaining 24 bits are used for the scan path. The outputs are 8, 6, and 24 for *data_bus_out*, *ad_bus*, and the scan path, respectively.

The testable model of the Adding Machine was obtained by putting together the circuit flip-flops in one chain. Figure 7.53 shows the virtual tester for this full scan testing of the testable netlist of this circuit. Other than the circuit under test being different and the number of shifts in the scan flip-flops, this tester is no different from that of the Residue-5 circuit discussed in Sect. 7.3. Figure 7.53 only shows the loop that is responsible for applying test vectors to the scan-inserted netlist.

The testbench shown reads 64 test vectors from *testFile* and applies them to the full scan circuit for every 976 circuit faults. Eight bits of every test vector are applied in parallel and the other 24 are shifted-in serially by the **repeat** loop of Fig. 7.53. This loop also shifts out 24 output bits. The **if** statement shown in Fig. 7.53 compares concatenation of the serially collected output and 14 parallel outputs with the 38 bits of the expected outputs from *testFile*.

7.5.2 RTL Design Multiple Scan

This section shows implementation of a multiple scan design in the Adding Machine of Chap. 2. As in the previous discussion on full scan, although the circuit diagram looks different than the Huffman model of Fig. 7.44, the circuit is still structured as such, and the multiple scan method of Sect. 7.4 can be applied to this circuit.

```

module    Tester;
    . . .

    CPU_ScanInserted FUT(clk, PI, PO, NbarT, Si, So);
    always #200 clk = ~clk;

    initial begin
        . . .
        while( !$feof(faultFile)) begin
            . . .
            while((!$feof(testFile))&&(!detected))begin
                status = $fscanf(testFile,"%b\n",line);

                pre_expected_st = cur_expected_st;

                expected_PO      = line[out_size - 1:0];
                cur_expected_st = line[st_size + out_size -1 :out_size];
                PI = line[st_size+out_size+in_size-1:st_size+out_size];

                NbarT = 1'b1;
                #delay;

                index = stIndex;
                repeat(24) begin
                    si      = line[index];
                    @(posedge clk);
                    saved_st[index - stIndex] = so;
                    index    = index + 1;
                end

                NbarT = 1'b0;
                @(posedge clk);
                SampledPO = PO;

                if({pre_expected_st, expected_PO} != {saved_st, SampledPO})
                begin
                    numOfDetected = numOfDetected + 1;
                    detected = 1;
                end
                #5;
            end//test
            . . .
        end//fault
        . . .
    end//initial
endmodule

```

Fig. 7.53 Full scan Adding Machine virtual tester

We use three equal scan chains: one covers *AC*, second covers *IR*, and the third covers all of *PC* plus two control flip-flops. Since the size of the scan chains is the same, we can use multiple parallel scan chains that simplifies scan controls. Figure 7.54 shows the three scan chains in the RTL diagram of the Adding Machine.

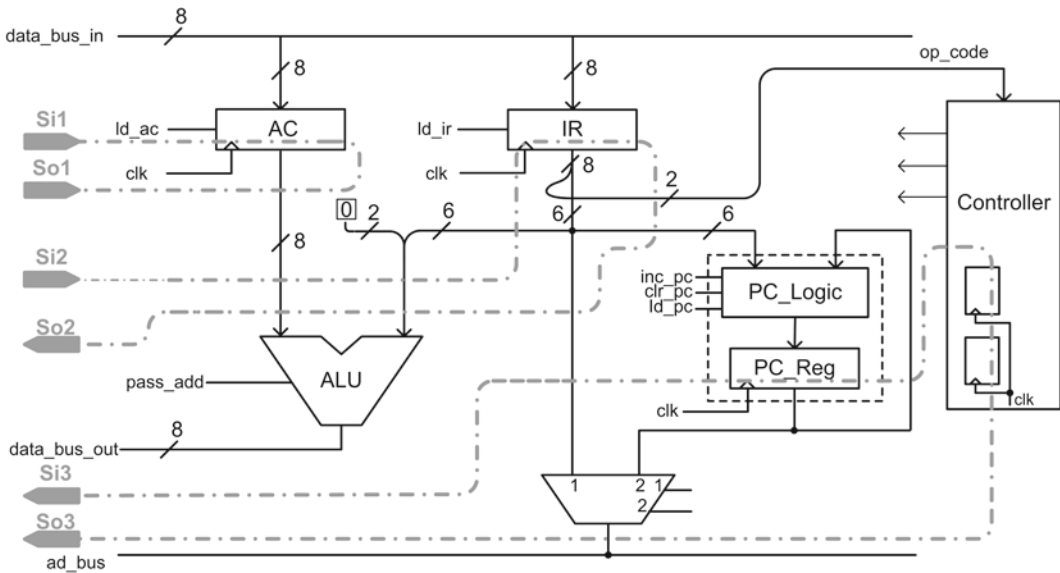


Fig. 7.54 Multiple parallel scans for Adding Machine

```

module CPU_M_ScanInserted(clk, {reset, data_bus_in}, {adr_bus,rd_mem,
                                wr_mem,data_bus_out}, NbarT, ir_Si, ac_Si,
                                pc_Si, ir_So, ac_So, cntrl_So);
    . . .

    [ dff INS_1 (wire_6,wire_96,clk,1'b0,1'b0,ir_en,NbarT,ir_Si, 1'b0);
      dff INS_2 (wire_2,wire_98,clk,1'b0,1'b0,ir_en,NbarT,wire_6, 1'b0);
      . . .
      dff INS_8 (wire_26,wire_173,clk,1'b0,1'b0,ir_en,NbarT,wire_54, 1'b0);
      //
      dff INS_9 (wire_130,wire_180,clk,1'b0,1'b0,ac_en,NbarT,ac_Si, 1'b0);
      dff INS_10(wire_135,wire_186,clk,1'b0,1'b0,ac_en,NbarT,wire_130, 1'b0);
      . . .
      dff INS_16(wire_52,wire_208,clk,1'b0,1'b0,ac_en,NbarT,wire_50, 1'b0);
      //
      dff INS_17 (wire_65,wire_211,clk,1'b0,1'b0,pc_en,NbarT,pc_Si, 1'b0);
      dff INS_18(wire_71,wire_216,clk,1'b0,1'b0,pc_en,NbarT,wire_65, 1'b0);
      . . .
      dff INS_24 (wire_4,wire_249,clk,1'b0,1'b0,pc_en,NbarT,wire_18, 1'b0);

      assign ir_So = wire_26;
      assign ac_So = wire_52;
      assign cntrl_So = wire_4;
endmodule

```

Fig. 7.55 Inserting three scan chains

The scan chains are manually inserted in the netlist that is the result of synthesizing the Verilog code of the Adding Machine. Insertion of scan makes changes in the ports of the netlist and arrangement of its flip-flops. Figure 7.55 shows partial code of the modified netlist.

As shown, *NbarT* and three serial inputs (*ir_Si*, *ac_Si*, and *pc_Si*) are added to the inputs, and three serial scan outputs (*ir_So*, *ac_So*, and *cntrl_So*) are added to the output ports of the scan-inserted netlist. The three bracketed sections in Fig. 7.55 show the three scan chains. All

```

module    Tester;

    . . .
    CPU_M_ScanInserted FUT(clk, PI, PO, NbarT, ir_Si, ac_Si, pc_Si, ir_So,
                           ac_So, cntrl_So);

    always #200 clk = ~clk;

    initial begin
        while( !$feof(faultFile)) begin
            . . .
            while((!$feof(testFile))&&(!detected)) begin
                status = $fscanf(testFile,"%b\n",line);
                pre_expected_st = cur_expected_st;
                expected_PO = line[out_size - 1:0];
                cur_expected_st = line[st_size + out_size - 1 :out_size];
                PI = line[st_size+out_size+in_size-1:st_size+out_size];
                NbarT = 1'b1;
                #delay;

                index = stIndex;
                repeat(8) begin
                    ir_Si = line[index];
                    ac_Si = line[index+8];
                    pc_Si = line[index+16];
                    @(posedge clk);
                    saved_st[index - stIndex] = ir_So;
                    saved_st[index+8 - stIndex] = ac_So;
                    saved_st[index+16 - stIndex] = cntrl_So;
                    index = index + 1;
                end

                NbarT = 1'b0;
                @(posedge clk);
                SampledPO = PO;

                if(({pre_expected_st, expected_PO} != {saved_st, SampledPO}))
                begin
                    numOfDetected = numOfDetected + 1;
                    detected = 1;
                end
                #5;
            end//test
        end//fault
    end//initial
endmodule

```

Fig. 7.56 Test program for multiple parallel scans

flip-flops use *NbarT* as input. The first chain begins with *ir_Si* and ends with *wire_26* (*INS_1* to *INS_8*) that becomes the *ir_So* output (see *assign* statements near the end of the code).

The serial input of the second scan is *ac_Si*, and its serial output is *wire_52* (*INS_9* to *INS_16*) that becomes *ac_So*. Finally, the third scan chain begins with *pc_Si* and ends with *cntrl_So* (*INS_17* to *INS_24*).

For developing a test program for this scan design, we have to consider that our design has three equal-length scan chains using the same clock, and the same normal/test mode (*NbarT*) control input. A virtual tester imitating the behavior of an ATE and its test program for testing our design is shown in Fig. 7.56. This figure only shows instantiation of the three-scan-chain circuit and the procedure

for application of test data. Other parts of the testbench that are basically the same as code discussed in Sect. 7.3 are not shown here. As shown, instantiation of the netlist, the partial code of which is shown in Fig. 7.55, includes *NbarT* as well as serial inputs and outputs of the three scan chains.

The test procedure is enclosed in the **initial** block shown in Fig. 7.56. All but the **repeat** loop of this procedure is the same as that of Fig. 7.53. The **repeat** loop here takes every 8 bits of *line* that has the input vector and assigns them to *ir_Si*, *ac_Si*, and *pc_Si*. This happens while *NbarT* for all three chains is 1 (shift mode). After the three assignments, the registers are clocked (simultaneously), after which outputs from *ir_So*, *ac_So*, and *cntrl_So* are collected in *saved_st*. These output values are positioned in the *saved_st* vector, 8 bits apart to allow output bits of the same chain to be adjacent to each other.

7.5.3 Scan Designs for RTL

It is easy to plan various scan designs for an RT level design by considering its RTL view. Once planning is done, an estimate for the test time can be obtained. The implementation of the planned scan design is done on the synthesized output of the RT-level HDL code. Developing a test program is made easier when thinking of the circuit in terms of its Huffman model after scan insertion.

Other possibilities for scan design for our RTL example of this section are partial scan, and multiple independent scans. Alternatively, if we partition the circuit into its datapath and a controller, separate scan design can be implemented for each part. Splitting a circuit into several parts can be implemented by isolated scan.

When planning different scan strategies, the number of test vectors, length of each test vector, and fault coverage are the main parameters to consider.

7.6 Summary

This chapter discussed DFT techniques. We started with some ad hoc methods that evolved into isolated scan. All such methods apply to combinational as well as sequential circuits. The scan design, that we started discussing in Sect. 7.3, was primarily focusing on sequential circuits, and its main purpose was to turn a sequential circuit into a combinational circuit, such that combinational TG methods could be used for generating tests.

Other scan designs that we discussed basically followed the same rule. In partial scan methods, the introduction of pseudo inputs and pseudo outputs does not completely turn a sequential model into a combinational one, and registers still exist in the obtained model. In spite of this, the obtained model can still be used for combinational test generation. The deviation from a pure combinational model affects the test procedure where extra clock pulses compensate for registers that are left in the model.

To show application of scan methods to RT-level designs and demonstrate the applicability of such schemes, the last section used a small RT-level design and a Verilog testbench to test its inserted scan. Larger designs can be partitioned into several subcomponents, where each subcomponent may require a different form of scan. Separating a design into its subcomponents can be done by a shift-register for scanning in and scanning out signal values in the interface of the subcomponents.

References

1. Abramovici M, Breuer MA, Friedman AD (1994) Digital systems testing and testable design. IEEE Press, Piscataway, NJ, revised printing.
2. Wilkins BR (1986) Testing digital circuits, an introduction. Van Nostrand Reinhold, Berkshire, UK.
3. Eichelberger EB, Lindbloom E, Waicukauski JA, Williams TW (1991) Structured logic testing. Prentice-Hall, Englewood Cliffs, NJ.
4. Agrawal VD, Mercer MR (1982) Testability measures – What do they tell us? In: Proceedings of the International Test Conference, Nov 1982, pp 391–396.
5. Willaims MJY, Angell JB (1973) Enhancing testability of large-scale integrated circuits via test points and additional logic. IEEE Trans Comput C-22(1):46–60.
6. Miczo A (2003) Digital logic testing and simulation, 2nd edn. Wiley, Hoboken, NJ.
7. Cheng K-T, Lin C-J (1995) Timing-driven test point insertion for full-scan and partial-scan bist. In: Proceedings of the International Test Conference, Oct 1995, pp 506–514.
8. Cheng K-T, Agrawal VD (1990) A partial scan method for sequential circuits with feedback. IEEE Trans Comput 39(4):544–548.
9. Narayanan S, Gupta R, Breuer MA (1993) Optimal configuring of multiple scan chains. IEEE Trans Comput 42(9):1121–1131.
10. Jha NK, Gupta S (2003) Testing of digital systems, Cambridge University Press, Cambridge, UK.